

بسم الله الرحمن الرحيم

بسمه تعالی



دانشگاه بناب  
گروه علوم کامپیوتر

پایان نامه کارشناسی رشته علوم کامپیوتر

شبکه‌های عصبی آگاه از فیزیک برای حل مسائل  
مستقیم و معکوس  
معادلات دیفرانسیل جزئی غیرخطی

استاد راهنما: دکتر بابک آذرنوید

پژوهشگر: محمد امیر بابازاد

شماره دانشجویی: ۱۴۰۲۱۳۵۱۰۴

شهریور ۱۴۰۵

**کلیه حقوق مادی و معنوی مرتبط با نتایج مطالعات، ابتکارات و  
نوآوری‌های  
ناشی از تحقیق موضوع این پایان‌نامه متعلق به دانشگاه بناب است.**

## تقديم به

پدر و مادر عزيزم؛

که هرچه دارم از دعاى خير و از خودگذشتگى آنهاست،  
و به همه آنان که چراغ دانش را در اين سرزمين روشن نگه داشته‌اند.

## سپاسگزاری

سپاس بی‌کران خداوند یکتا را که توفیق انجام این پژوهش را به این‌جانب عطا فرمود. بر خود لازم می‌دانم از استاد ارجمند، جناب آقای دکتر بابک آذرنوید که راهنمایی این پایان‌نامه را بر عهده داشتند و در تمام مراحل انجام آن، از انتخاب موضوع تا تدوین نهایی، با حوصله و دقت راهنمایم بودند، صمیمانه قدردانی کنم. بی‌شک بدون رهنمودها و حمایت‌های ایشان، این پژوهش به سرانجام نمی‌رسید. همچنین از همه استادان گروه علوم کامپیوتر دانشگاه بناب که در دوران تحصیل از دانش و تجربه آنان بهره‌مند شدم، تشکر و قدردانی می‌کنم. در پایان، از پدر و مادر مهربانم که همواره مشوق و پشتیبان من در مسیر علم‌آموزی بوده‌اند و از همه دوستان و عزیزانی که به هر شکل مرا در انجام این پژوهش یاری رساندند، سپاسگزارم.

محمد امیر بابازاد

شهریور ۱۴۰۵

## چکیده

حل عددی معادلات دیفرانسیل جزئی غیرخطی از مسائل بنیادی محاسبات علمی است. روش‌های عددی کلاسیک هرچند دقیق و پرکاربردند، به شبکه محاسباتی وابسته‌اند و در ابعاد بالا با مشکل مواجه می‌شوند؛ از سوی دیگر، مدل‌های یادگیری عمیق صرفاً داده‌محور به حجم زیادی داده آموزشی نیاز دارند و سازگاری خروجی آن‌ها با قوانین فیزیک تضمین شده نیست. در سال‌های اخیر، چارچوب شبکه‌های عصبی آگاه از فیزیک به‌عنوان راهی برای ترکیب دانش فیزیکی (معادله حاکم بر سیستم) با یادگیری ماشین معرفی شده است. هدف این پایان‌نامه، مطالعه، پیاده‌سازی و ارزیابی این چارچوب برای حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی است.

در این پژوهش، ابتدا مبانی نظری لازم شامل معادلات دیفرانسیل جزئی، روش‌های عددی کلاسیک، شبکه‌های عصبی، مشتق‌گیری خودکار و پیشینه یادگیری ماشین آگاه از فیزیک مرور شد. سپس چارچوب شبکه آگاه از فیزیک شامل تعریف شبکه باقیمانده، تابع هزینه ترکیبی داده و فیزیک، و مدل‌های زمان‌پیوسته و زمان‌گسسته تشریح گردید. در بخش عملی، مدل زمان‌پیوسته برای معادله برگرز یک‌بعدی با زبان پایتون و کتابخانه جکس پیاده‌سازی شد و یک حل‌گر مرجع مستقل بر پایه روش طیفی فوری و گام‌زنی زمانی مرتبه چهار برای تولید جواب دقیق توسعه یافت و با جواب نیمه‌تحلیلی کول - هوپف راستی‌آزمایی شد.

در مسئله مستقیم، جواب کل دامنه زمانی - مکانی تنها با ۲۰۰ نقطه داده اولیه و مرزی بازسازی شد و خطای نسبی  $2.15 \times 10^{-2}$  در نرم  $L_2$  به دست آمد؛ خطا عمدتاً در لایه باریک جبهه موج متمرکز بود. در مسئله معکوس، ضرایب معادله از روی ۵۰۰۰ مشاهده پراکنده شناسایی شدند: ضریب انتقال با خطای ۸.۸٪ و ضریب پخش با خطای ۳۲.۶٪ در حالت بدون نویز، و به‌ترتیب با خطای ۷.۷٪ و ۲۴.۹٪ در حضور یک درصد نویز. نتایج نشان می‌دهد که این چارچوب با داده اندک به جوابی قابل قبول می‌رسد، در برابر نویز پایداری قابل قبولی دارد و گلوگاه اصلی آن، تفکیک لایه‌های تند جواب و حساسیت ذاتی شناسایی ضرایب کوچک است.

**کلیدواژه‌ها:** شبکه عصبی آگاه از فیزیک، معادله دیفرانسیل جزئی غیرخطی، مسئله مستقیم، مسئله معکوس، معادله برگرز، مشتق‌گیری خودکار

## فهرست علائم اختصاری

در این پایان نامه به منظور سهولت و کوتاه تر شدن متن، از علائم اختصاری زیر استفاده شده است:

| علامت اختصاری | شرح                        |
|---------------|----------------------------|
| PINN          | شبکه عصبی آگاه از فیزیک    |
| PDE           | معادله دیفرانسیل جزئی      |
| ODE           | معادله دیفرانسیل معمولی    |
| ANN           | شبکه عصبی مصنوعی           |
| MLP           | پرسپترون چندلایه           |
| MSE           | میانگین مربعات خطا         |
| SSE           | مجموع مربعات خطا           |
| IC            | شرط اولیه                  |
| BC            | شرط مرزی                   |
| RK            | رانگ - کوتا                |
| L-BFGS        | روش شبه نیوتنی حافظه محدود |
| ReLU          | واحد خطی یکسوشده           |
| GPU           | واحد پردازش گرافیکی        |
| CPU           | واحد پردازش مرکزی          |

# فهرست مطالب

|    |                                           |
|----|-------------------------------------------|
| پ  | سیاسگزاری                                 |
| ت  | چکیده                                     |
| ث  | فهرست علائم اختصاری                       |
| ج  | فهرست شکل ها                              |
| ح  | فهرست جدول ها                             |
| ۱  | ۱ مقدمه و کلیات پژوهش                     |
| ۲  | ۱-۱ موضوع پژوهش                           |
| ۲  | ۲-۱ بیان مسئله                            |
| ۳  | ۳-۱ اهداف پژوهش                           |
| ۳  | ۴-۱ سؤال های پژوهش                        |
| ۴  | ۵-۱ روش انجام پژوهش                       |
| ۴  | ۶-۱ ساختار پایان نامه                     |
| ۶  | ۲ مبانی نظری و پیشینه پژوهش               |
| ۶  | ۱-۲ معادلات دیفرانسیل جزئی                |
| ۸  | ۲-۲ روش های عددی کلاسیک                   |
| ۹  | ۳-۲ شبکه های عصبی مصنوعی                  |
| ۱۱ | ۴-۲ آموزش شبکه های عمیق                   |
| ۱۲ | ۵-۲ مشتق گیری خودکار                      |
| ۱۳ | ۶-۲ پیشینه پژوهش                          |
| ۱۴ | ۷-۲ جمع بندی فصل                          |
| ۱۵ | ۳ شبکه های عصبی آگاه از فیزیک             |
| ۱۵ | ۱-۳ صورت کلی مسئله و ایده محوری           |
| ۱۶ | ۲-۳ مدل های زمان پیوسته برای مسئله مستقیم |
| ۱۷ | ۳-۳ مدل های زمان گسسته                    |
| ۱۸ | ۴-۳ مسئله معکوس و کشف معادله              |
| ۱۹ | ۵-۳ نکات عملی در پیاده سازی و آموزش       |
| ۲۰ | ۶-۳ جمع بندی فصل                          |
| ۲۱ | ۴ پیاده سازی و ارزیابی نتایج              |
| ۲۱ | ۱-۴ محیط و ابزار پیاده سازی               |
| ۲۱ | ۲-۴ حل گر مرجع و راستی آزمایی آن          |
| ۲۲ | ۳-۴ حل مستقیم معادله برگرز                |
| ۲۴ | ۴-۴ کشف پارامترهای معادله برگرز           |
| ۲۶ | ۵-۴ بحث و تحلیل نتایج                     |



|    |                                              |
|----|----------------------------------------------|
| ۲۷ | ۶-۴ جمع‌بندی فصل . . . . .                   |
| ۲۹ | ۵ نتیجه‌گیری و پیشنهادها                     |
| ۲۹ | ۱-۵ خلاصه پژوهش . . . . .                    |
| ۲۹ | ۲-۵ یافته‌ها و نتیجه‌گیری . . . . .          |
| ۳۰ | ۳-۵ پیشنهادها برای پژوهش‌های آینده . . . . . |
| ۳۲ | فهرست منابع                                  |
| ۳۴ | واژه‌نامه                                    |
| ۳۶ | پیوست الف: کد پیاده‌سازی                     |
| ۵۱ | فرم ارزیابی پایان‌نامه                       |
| ۵۲ | Abstract                                     |

## فهرست تصاویر

|     |                                                                                                                   |    |
|-----|-------------------------------------------------------------------------------------------------------------------|----|
| ۱-۲ | نمایی شماتیک از یک پرسپترون چندلایه با دو ورودی، دو لایه پنهان و یک خروجی                                         | ۱۰ |
| ۱-۳ | نمای کلی چارچوب شبکه عصبی آگاه از فیزیک: شبکه جواب، مشتق‌گیری خودکار، شبکه باقیمانده و حلقه آموزش                 | ۱۶ |
| ۱-۴ | توزیع نقاط آموزشی در مسئله مستقیم: نقاط داده روی شرط اولیه و مرزها و نقاط هم‌مکانی در دامنه                       | ۲۳ |
| ۲-۴ | نمودار تاریخچه تابع هزینه در مسئله مستقیم: کاهش سریع در مرحله آدام و بهبود نهایی با روش شبه‌نیوتنی                | ۲۴ |
| ۳-۴ | مقایسه جواب دقیق و پیش‌بینی شبکه آگاه از فیزیک در سه لحظه زمانی در مسئله مستقیم معادله برگرز                      | ۲۴ |
| ۴-۴ | نقشه خطای مطلق بین جواب دقیق و پیش‌بینی شبکه در دامنه زمانی - مکانی؛ خطا عمدتاً در لایه باریک جبهه موج متمرکز است | ۲۵ |
| ۵-۴ | توزیع ۵۰۰۰ نقطه مشاهده پراکنده مورد استفاده در مسئله معکوس؛ رنگ نقاط نشان‌دهنده مقدار جواب است                    | ۲۶ |

## فهرست جداول

|     |                                                                 |    |
|-----|-----------------------------------------------------------------|----|
| ۱-۲ | مقایسه اجمالی روش‌های عددی کلاسیک حل معادلات دیفرانسیل جزئی     | ۹  |
| ۲-۲ | توابع فعال‌ساز پرکاربرد در شبکه‌های عصبی                        | ۱۰ |
| ۱-۳ | مقایسه مدل‌های زمان‌پیوسته و زمان‌گسسته در چارچوب آگاه از فیزیک | ۲۰ |
| ۱-۴ | تنظیمات آموزش مدل زمان‌پیوسته در مسئله مستقیم                   | ۲۳ |
| ۲-۴ | مقایسه نتیجه مسئله مستقیم با مقاله مرجع                         | ۲۵ |
| ۳-۴ | نتایج شناسایی پارامترهای معادله برگرز در مسئله معکوس            | ۲۶ |

# فصل ۱

## مقدمه و کلیات پژوهش

بخش بزرگی از پدیده‌هایی که در علوم و مهندسی با آن‌ها سروکار داریم، از حرکت سیالات و انتقال حرارت گرفته تا مکانیک کوانتومی و انتشار موج، با معادلات دیفرانسیل جزئی<sup>۱</sup> توصیف می‌شوند. به همین دلیل، حل دقیق و سریع این معادلات همواره یکی از مسائل بنیادی در محاسبات علمی بوده است. از سوی دیگر، در سال‌های اخیر یادگیری ماشین و به‌ویژه یادگیری عمیق<sup>۲</sup> موفقیت‌های چشمگیری در حوزه‌هایی مانند بینایی ماشین و پردازش زبان به دست آورده است، اما به‌کارگیری مستقیم این روش‌ها در مسائل فیزیکی معمولاً با چالش‌هایی مانند نیاز به داده آموزشی فراوان و ناسازگاری پاسخ با قوانین فیزیک روبه‌روست. همین مسئله پژوهشگران را به این فکر انداخته است که دانش فیزیکی مسئله، یعنی همان معادله دیفرانسیل حاکم بر سیستم، را مستقیماً وارد فرایند یادگیری کنند. حاصل این ایده، خانواده‌ای از روش‌ها با عنوان شبکه‌های عصبی آگاه از فیزیک<sup>۳</sup> است که در سال ۲۰۱۹ با مقاله رئیسی و همکارانش به‌طور جدی معرفی شد [۱] و به‌سرعت به یکی از پراستنادترین موضوعات در مرز مشترک یادگیری ماشین و محاسبات علمی تبدیل گردید.

در این پایان‌نامه، چارچوب شبکه‌های عصبی آگاه از فیزیک برای حل مسائل مستقیم و معکوس شامل معادلات دیفرانسیل جزئی غیرخطی بررسی می‌شود. مسئله نمونه‌ای که در بخش عملی این پژوهش انتخاب شده، معادله برگرز<sup>۴</sup> است؛ معادله‌ای کلاسیک که به‌سبب رفتار غیرخطی و تشکیل جبهه‌های تند، معیاری شناخته‌شده برای سنجش روش‌های عددی به شمار می‌رود. افزون بر مطالعه نظری، یک پیاده‌سازی کامل از مدل زمان‌پیوسته با زبان پایتون و کتابخانه جکس<sup>۵</sup> انجام و نتایج آن با حل مرجع به دست آمده از یک حل‌گر طیفی مستقل مقایسه شده است.

---

<sup>1</sup>Partial Differential Equation (PDE)

<sup>2</sup>Deep Learning

<sup>3</sup>Physics-Informed Neural Network (PINN)

<sup>4</sup>Burgers Equation

<sup>5</sup>JAX

## ۱-۱ موضوع پژوهش

موضوع این پایان‌نامه «شبکه‌های عصبی آگاه از فیزیک برای حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی» است. در مسئله مستقیم<sup>۶</sup>، پارامترهای معادله معلوم فرض می‌شوند و هدف، یافتن جواب معادله با استفاده از تعداد اندکی داده اولیه و مرزی است. در مسئله معکوس<sup>۷</sup>، برعکس، از روی مشاهده‌های پراکنده و احیاناً نویزی جواب، مقصود کشف پارامترهای مجهول معادله حاکم بر سیستم است. هر دو دسته مسئله در مقاله مرجع این پایان‌نامه [۱] بررسی شده‌اند و ما نیز هر دو را، هم از نظر نظری و هم در قالب پیاده‌سازی عملی، دنبال می‌کنیم.

نکته مهم در این چارچوب، نقش مشتق‌گیری خودکار<sup>۸</sup> است: مشتق‌های جواب نسبت به متغیرهای مکان و زمان، به‌جای استفاده از شبکه محاسباتی و تفاضلات محدود، مستقیماً از روی گراف محاسباتی شبکه عصبی و با دقت ماشین محاسبه می‌شوند. به این ترتیب، باقیمانده معادله دیفرانسیل به‌صورت یک شبکه عصبی جدید درمی‌آید که در همان تابع هزینه آموزش وارد می‌شود و شبکه را وامی‌دارد پاسخی بیاموزد که هم به داده‌ها نزدیک باشد و هم معادله را ارضا کند.

## ۲-۱ بیان مسئله

روش‌های عددی کلاسیک مانند تفاضل‌های محدود<sup>۹</sup>، المان‌های محدود<sup>۱۰</sup> و روش‌های طیفی<sup>۱۱</sup> دهه‌هاست که ابزار اصلی حل معادلات دیفرانسیل جزئی هستند و مبانی نظری آن‌ها به‌خوبی توسعه یافته است [۱۶]. با این حال، این روش‌ها معمولاً به تولید شبکه محاسباتی وابسته‌اند، در ابعاد بالا با پدیده نفرین ابعاد<sup>۱۲</sup> مواجه می‌شوند و برای هر هندسه یا شرط مرزی جدید باید از نو اجرا شوند. از طرفی، مدل‌های یادگیری عمیق صرفاً داده‌محور هرچند در تقریب توابع بسیار توانمندند، اما برای آموزش به حجم زیادی داده برچسب‌خورده نیاز دارند؛ داده‌ای که در بسیاری از مسائل فیزیکی و مهندسی یا اصلاً در دسترس نیست یا تهیه آن بسیار پرهزینه است. علاوه بر این، خروجی چنین مدل‌هایی هیچ تضمینی برای سازگاری با قوانین فیزیک ندارد و ممکن است در خارج از

<sup>۶</sup>Forward Problem

<sup>۷</sup>Inverse Problem

<sup>۸</sup>Automatic Differentiation

<sup>۹</sup>Finite Difference

<sup>۱۰</sup>Finite Element

<sup>۱۱</sup>Spectral Methods

<sup>۱۲</sup>Curse of Dimensionality

محدوده داده آموزشی رفتار غیرفیزیکی از خود نشان دهد. مسئله‌ای که این پایان‌نامه به آن می‌پردازد را می‌توان چنین خلاصه کرد: چگونه می‌توان دانش پیشینی موجود درباره یک سیستم فیزیکی، یعنی معادله دیفرانسیل جزئی حاکم بر آن، را به صورت اصولی وارد آموزش یک شبکه عصبی کرد، به گونه‌ای که اولاً با داده آموزشی اندک بتوان جواب دقیق مسئله را به دست آورد و ثانیاً از روی مشاهده‌های محدود، پارامترهای مجهول خود معادله را نیز شناسایی کرد؟ پاسخ مقاله مرجع به این پرسش، معرفی شبکه‌های عصبی آگاه از فیزیک در دو گونه مدل زمان‌پیوسته و زمان‌گسسته است [۱] که مبنای نظری و عملی این پژوهش را تشکیل می‌دهد.

### ۳-۱ اهداف پژوهش

هدف اصلی این پژوهش، مطالعه، پیاده‌سازی و ارزیابی چارچوب شبکه‌های عصبی آگاه از فیزیک برای حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی است. برای دستیابی به این هدف اصلی، اهداف فرعی زیر دنبال می‌شوند:

۱. مطالعه مبانی نظری لازم شامل معادلات دیفرانسیل جزئی، روش‌های عددی کلاسیک، شبکه‌های عصبی، بهینه‌سازی و مشتق‌گیری خودکار؛
۲. بررسی دقیق مقاله مرجع و استخراج مدل‌های زمان‌پیوسته و زمان‌گسسته برای مسائل مستقیم و معکوس؛
۳. پیاده‌سازی یک حل‌گر مرجع مستقل بر پایه روش طیفی فوریه برای معادله برگرز به منظور تولید جواب دقیق جهت مقایسه؛
۴. پیاده‌سازی مدل زمان‌پیوسته شبکه آگاه از فیزیک برای حل مستقیم معادله برگرز و سنجش دقت آن نسبت به حل مرجع؛
۵. پیاده‌سازی مسئله معکوس برای کشف ضرایب معادله برگرز از روی داده‌های پراکنده و بررسی اثر نویز بر دقت شناسایی پارامترها.

### ۴-۱ سؤال‌های پژوهش

سؤال‌هایی که این پژوهش در پی پاسخ‌گویی به آن‌هاست عبارت‌اند از:

۱. آیا می‌توان با یک شبکه عصبی نسبتاً کوچک و تنها با استفاده از داده‌های اولیه و مرزی، جواب دقیق یک معادله دیفرانسیل جزئی غیرخطی مانند معادله برگرز را

- در کل دامنه زمانی - مکانی بازسازی کرد؟
۲. سازوکار وارد کردن معادله دیفرانسیل در تابع هزینه شبکه چگونه است و مشتق‌گیری خودکار در این میان چه نقشی ایفا می‌کند؟
۳. دقت جواب به دست آمده از شبکه آگاه از فیزیک در مقایسه با یک حل‌گر عددی استاندارد چقدر است و خطا در کدام نواحی دامنه بیشتر می‌شود؟
۴. آیا می‌توان ضرایب مجهول معادله را نیز هم‌زمان با جواب، از روی مشاهده‌های پراکنده و نویزی فراگرفت و دقت این شناسایی چه میزان است؟
۵. مدل‌های زمان‌پیوسته و زمان‌گسسته هر یک برای چه دسته‌ای از مسائل مناسب‌ترند و چه محدودیت‌هایی دارند؟

## ۵-۱ روش انجام پژوهش

این پژوهش از نظر روش، ترکیبی از مطالعه نظری و پیاده‌سازی تجربی است و در مراحل زیر انجام شده است. نخست، مبانی نظری موضوع با مطالعه منابع معتبر در زمینه معادلات دیفرانسیل جزئی [۱۷]، روش‌های عددی [۱۶] و یادگیری عمیق [۱۱] مرور شد. سپس مقاله مرجع [۱] به‌طور کامل مطالعه و ایده‌های اصلی آن، یعنی تعریف شبکه باقیمانده، طراحی تابع هزینه ترکیبی و دو گونه مدل زمان‌پیوسته و زمان‌گسسته، استخراج گردید.

در مرحله بعد، معادله برگرز یک‌بعدی به‌عنوان مسئله نمونه انتخاب شد؛ زیرا هم در مقاله مرجع به‌تفصیل بررسی شده و امکان مقایسه نتایج وجود دارد و هم به‌سبب رفتار غیرخطی و تشکیل گرادیان‌های تند، مسئله‌ای چالش‌برانگیز و در عین حال قابل‌فهم برای سطح کارشناسی است. برای ارزیابی مستقل نتایج، یک حل‌گر مرجع با روش طیفی فوریه و گام‌زنی زمانی نمایی مرتبه چهار پیاده‌سازی شد. آنگاه مدل زمان‌پیوسته برای مسئله مستقیم و نیز مسئله معکوس کشف پارامترها با زبان پایتون و کتابخانه جکس پیاده‌سازی و آموزش داده شد. در پایان، نتایج عددی شامل دقت جواب، همگرایی آموزش و خطای شناسایی پارامترها تحلیل و با نتایج گزارش‌شده در مقاله مرجع مقایسه گردید.

## ۶-۱ ساختار پایان‌نامه

ادامه این پایان‌نامه به شرح زیر سازماندهی شده است. در فصل دوم، مبانی نظری و پیشینه پژوهش شامل معادلات دیفرانسیل جزئی، روش‌های عددی کلاسیک، شبکه‌های

عصبی، مشتق‌گیری خودکار و مروری بر کارهای پیشین در زمینه یادگیری ماشین آگاه از فیزیک ارائه می‌شود. فصل سوم به تشریح روش پژوهش، یعنی چارچوب شبکه‌های عصبی آگاه از فیزیک، مدل‌های زمان‌پیوسته و زمان‌گسسته و فرمول‌بندی مسائل مستقیم و معکوس اختصاص دارد. در فصل چهارم، جزئیات پیاده‌سازی و نتایج عددی برای معادله برگرز، شامل حل مستقیم و کشف پارامترها، گزارش و تحلیل می‌شود. سرانجام در فصل پنجم، خلاصه‌ای از یافته‌ها، نتیجه‌گیری و پیشنهادهایی برای ادامه پژوهش ارائه می‌گردد.



## فصل ۲

### مبانی نظری و پیشینه پژوهش

هدف این فصل، مرور مفاهیمی است که برای فهم چارچوب شبکه‌های عصبی آگاه از فیزیک لازم‌اند. ابتدا معادلات دیفرانسیل جزئی و چند مثال مهم آن‌ها معرفی می‌شوند و سپس روش‌های عددی کلاسیک حل این معادلات به اختصار بررسی می‌گردد. در ادامه، مبانی شبکه‌های عصبی و آموزش آن‌ها مرور می‌شود و پس از آن به مشتق‌گیری خودکار، به عنوان ابزار کلیدی این پایان‌نامه، می‌پردازیم. در پایان فصل نیز پیشینه پژوهش در زمینه ترکیب یادگیری ماشین با دانش فیزیک مرور می‌شود تا جایگاه مقاله مرجع این پایان‌نامه مشخص گردد.

#### ۱-۲ معادلات دیفرانسیل جزئی

معادله دیفرانسیل جزئی رابطه‌ای است میان یک تابع مجهول چندمتغیره و مشتق‌های جزئی آن. به طور کلی، یک معادله دیفرانسیل جزئی برای تابع مجهول  $u(x)$  را می‌توان به شکل زیر نوشت:

$$F(x, u, Du, D^2u, \dots, D^k u) = 0, \quad (1-2)$$

که در آن  $x$  بردار متغیرهای مستقل (مانند مکان و زمان)،  $Du$  شامل مشتق‌های مرتبه اول،  $D^2u$  شامل مشتق‌های مرتبه دوم و به همین ترتیب است. بزرگ‌ترین مرتبه مشتق ظاهرشده در معادله را مرتبه معادله می‌نامند. اگر تابع  $F$  نسبت به  $u$  و مشتق‌های آن خطی باشد، معادله خطی و در غیر این صورت غیرخطی<sup>۱</sup> نامیده می‌شود [۱۷].

برای آن‌که مسئله به خوبی تعریف شود، معمولاً معادله باید همراه با شرط اولیه و شرط مرزی در نظر گرفته شود. شرط اولیه مقدار جواب (و گاهی مشتق زمانی آن) را در لحظه شروع مشخص می‌کند و شرط مرزی رفتار جواب را روی مرز دامنه مکانی تعیین می‌نماید. مسئله‌ای که در آن جواب با این شرایط اضافی یکتا و نسبت به داده‌ها پایدار

---

<sup>۱</sup>Nonlinear

باشد، مسئله خوش‌وضع<sup>۲</sup> نامیده می‌شود.

در ادامه سه مثال کلاسیک را که در این پایان‌نامه نیز به آن‌ها اشاره خواهد شد معرفی می‌کنیم.

**معادله گرما.** معادله گرما ساده‌ترین مثال از معادلات سهموی است و توزیع دما در یک جسم را بر حسب زمان توصیف می‌کند. شکل یک‌بعدی آن چنین است:

$$u_t = \alpha u_{xx}, \quad (۲-۲)$$

که در آن  $u(t, x)$  دما،  $\alpha$  ضریب پخش و زیروندها نشان‌دهنده مشتق جزئی نسبت به زمان  $(t)$  و مکان  $(x)$  هستند.

**معادله برگرز.** معادله برگرز<sup>۳</sup> یک معادله غیرخطی است که می‌توان آن را ساده‌شده یک‌بعدی معادلات ناویر - استوکس دانست و به‌همین دلیل در دینامیک سیالات و نظریه ترافیک و غیره کاربرد دارد [۱۸]. شکل استاندارد آن عبارت است از:

$$u_t + uu_x - \nu u_{xx} = 0, \quad (۳-۲)$$

که در آن  $\nu$  ضریب لزجت است. جمله غیرخطی  $uu_x$  باعث تند شدن موج و جمله  $u_{xx}$  باعث پخش و هموار شدن آن می‌شود. رقابت این دو جمله، به‌ویژه به‌ازای  $\nu$ های کوچک، جبهه‌های بسیار تندی در جواب ایجاد می‌کند که محاسبه دقیق آن‌ها برای روش‌های عددی چالشبرانگیز است. به همین سبب، معادله برگرز معیاری استاندارد برای سنجش روش‌های جدید محسوب می‌شود و در این پایان‌نامه نیز مسئله نمونه پخش عملی است.

**معادله شرودینگر.** به‌عنوان مثالی از فیزیک کوانتوم می‌توان معادله شرودینگر غیرخطی را نام برد که در مقاله مرجع نیز یکی از مسائل نمونه است [۱]. این مثال نشان می‌دهد که چارچوب مورد بحث به یک حوزه خاص محدود نیست. در این پایان‌نامه با دو دسته مسئله سروکار داریم: در مسئله مستقیم، معادله و پارامترهای آن معلوم‌اند و جواب  $u$  مجهول است؛ در مسئله معکوس، بخشی از جواب به‌صورت مشاهده در اختیار است و هدف، تعیین پارامترهای مجهول معادله است. فصل سوم هر دو دسته را به‌تفصیل بررسی می‌کند.

---

<sup>۲</sup>Well-posed Problem

<sup>۳</sup>Burgers Equation

## ۲-۲ روش‌های عددی کلاسیک

از آنجا که تنها برای دسته محدودی از معادلات دیفرانسیل جزئی می‌توان جواب تحلیلی بسته یافت، روش‌های عددی ابزار اصلی حل این معادلات در عمل هستند. ایده مشترک همه این روش‌ها، تبدیل مسئله پیوسته به یک مسئله گسسته با تعداد متناهی مجهول است که با کامپیوتر قابل حل باشد [۱۶]. در ادامه سه خانواده مهم این روش‌ها را مرور می‌کنیم.

**روش تفاضل‌های محدود.** در این روش، دامنه با یک شبکه منظم از نقاط پوشانده می‌شود و مشتق‌ها با ترکیب خطی مقادیر تابع در نقاط مجاور تقریب زده می‌شوند. برای نمونه، مشتق دوم مکانی با فرمول مرکزی مرتبه دو چنین تقریب زده می‌شود:

$$u_{xx}(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2}, \quad (۴-۲)$$

که در آن  $\Delta x$  فاصله نقاط شبکه است. سادگی پیاده‌سازی مزیت اصلی این روش است، اما در هندسه‌های پیچیده و برای رسیدن به دقت بالا (که نیازمند شبکه بسیار ریز است) با مشکل مواجه می‌شود.

**روش المان‌های محدود.** در این روش، دامنه به قطعه‌های کوچکی به نام المان تقسیم می‌شود و جواب در هر المان با ترکیب چند تابع پایه ساده تقریب زده می‌شود. سپس با استفاده از فرمول‌بندی ضعیف<sup>۴</sup> معادله، یک دستگاه معادلات جبری برای ضرایب مجهول به دست می‌آید. قدرت اصلی این روش در برخورد با هندسه‌های پیچیده و شرایط مرزی متنوع است و به همین دلیل در نرم‌افزارهای مهندسی کاربرد گسترده‌ای دارد.

**روش‌های طیفی.** در روش‌های طیفی، جواب به صورت ترکیب خطی از توابع پایه سراسری مانند چندجمله‌ای‌ها یا توابع مثلثاتی نوشته می‌شود. برای مسائل با جواب هموار، این روش‌ها همگرایی بسیار سریعی (نمایی) دارند؛ یعنی با تعداد نقاط نسبتاً کم به دقت بالایی می‌رسند [۲۰]. در بخش عملی این پایان‌نامه نیز از یک روش طیفی بر پایه تبدیل فوریه برای تولید جواب مرجع معادله برگرز استفاده شده است.

مقایسه اجمالی این سه خانواده روش در جدول ۲-۱ آمده است. نکته مشترکی که همه روش‌های کلاسیک دارند، وابستگی به شبکه محاسباتی و نیاز به اجرای مجدد از ابتدا به ازای هر شرط اولیه یا پارامتر جدید است. همچنین در ابعاد بالا، تعداد نقاط شبکه به صورت نمایی رشد می‌کند که همان پدیده نفرین ابعاد است. این محدودیت‌ها انگیزه اصلی جست‌وجو برای روش‌های جایگزین، از جمله روش‌های مبتنی بر شبکه‌های

<sup>۴</sup>Weak Formulation

جدول ۲-۱: مقایسه اجمالی روش‌های عددی کلاسیک حل معادلات دیفرانسیل جزئی

| روش             | ایده اصلی                                         | مزیت مهم                           | محدودیت مهم                               |
|-----------------|---------------------------------------------------|------------------------------------|-------------------------------------------|
| تفاضل‌های محدود | تقریب مشتق با مقادیر نقاط مجاور روی شبکه منظم     | سادگی پیاده‌سازی و تحلیل           | دشواری هندسه‌های پیچیده                   |
| المان‌های محدود | تقسیم دامنه به المان و استفاده از فرمول‌بندی ضعیف | انعطاف در هندسه و شرط مرزی         | پیچیدگی پیاده‌سازی و هزینه محاسباتی       |
| روش‌های طیفی    | بسط جواب بر حسب توابع پایه سراسری                 | دقت بسیار بالا برای جواب‌های هموار | محدودیت هندسه‌های ساده و جواب‌های ناهموار |

عصبی، به شمار می‌رود.

## ۳-۲ شبکه‌های عصبی مصنوعی

شبکه عصبی مصنوعی<sup>۵</sup> مدلی محاسباتی است که از ساختار مغز الهام گرفته شده و از واحدهای ساده‌ای به نام نورون تشکیل می‌شود. هر نورون، ترکیب خطی ورودی‌های خود را محاسبه کرده و سپس یک تابع غیرخطی به نام تابع فعال‌ساز<sup>۶</sup> روی آن اعمال می‌کند:

$$y = \sigma \left( \sum_{i=1}^n w_i x_i + b \right), \quad (۵-۲)$$

که در آن  $x_i$  ورودی‌ها،  $w_i$  وزن‌ها،  $b$  بایاس و  $\sigma$  تابع فعال‌ساز است. اتصال لایه‌هایی از این نورون‌ها به یکدیگر، شبکه‌ای به نام پرسپترون چندلایه<sup>۷</sup> می‌سازد. خروجی لایه  $l$ ام چنین شبکه‌ای به صورت برداری چنین نوشته می‌شود:

$$\mathbf{h}^{(l)} = \sigma \left( \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right), \quad (۶-۲)$$

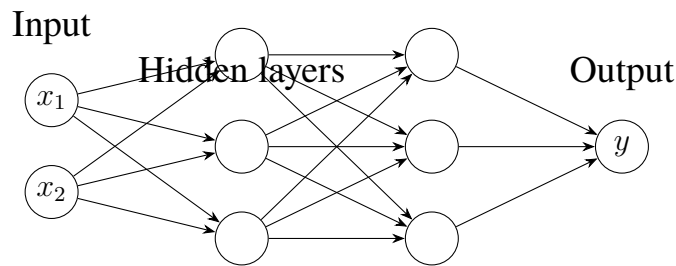
که  $\mathbf{h}^{(0)}$  همان بردار ورودی است. نمایی شماتیک از یک پرسپترون چندلایه با دو لایه پنهان در شکل ۲-۱ نشان داده شده است.

انتخاب تابع فعال‌ساز اثر مهمی بر رفتار شبکه دارد. پرکاربردترین توابع فعال‌ساز

<sup>۵</sup>Artificial Neural Network (ANN)

<sup>۶</sup>Activation Function

<sup>۷</sup>Multi-Layer Perceptron (MLP)



شکل ۱-۲: نمایی شماتیک از یک پرسپترون چندلایه با دو ورودی، دو لایه پنهان و یک خروجی

جدول ۲-۲: توابع فعال‌ساز پرکاربرد در شبکه‌های عصبی

| ویژگی                           | فرمول                                          | نام تابع          |
|---------------------------------|------------------------------------------------|-------------------|
| خروجی در بازه $(0, 1)$ ، هموار  | $\sigma(z) = \frac{1}{1 + e^{-z}}$             | سیگموئید          |
| خروجی در بازه $(-1, 1)$ ، هموار | $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ | تانژانت هیپربولیک |
| محاسبه سریع، نامشتق‌پذیر در صفر | $\text{ReLU}(z) = \max(0, z)$                  | واحد خطی یکسوشده  |

در جدول ۲-۲ آمده‌اند. در شبکه‌های آگاه از فیزیک معمولاً از تابع تانژانت هیپربولیک استفاده می‌شود، زیرا هموار و بی‌نهایت‌بار مشتق‌پذیر است و مشتق‌گیری‌های پی‌درپی نسبت به ورودی‌ها (که در فصل سوم خواهیم دید ضروری است) برای آن به‌خوبی تعریف می‌شود؛ در حالی که توابعی مانند ReLU در مبدأ مشتق ندارند و مشتق دوم آن‌ها تقریباً همه‌جا صفر است.

توانایی نظری شبکه‌های عصبی در تقریب توابع، با قضیه تقریب جهانی تضمین می‌شود. صورت ساده‌ای از این قضیه که توسط سیبنکو<sup>۸</sup> اثبات شده [۹] چنین بیان می‌شود:

**قضیه ۱.۲ (تقریب جهانی).** هر تابع پیوسته روی یک مجموعه فشرده را می‌توان با یک شبکه عصبی پیش‌خور با یک لایه پنهان و تابع فعال‌ساز غیرخطی مناسب، با هر دقت دلخواه تقریب زد.

البته این قضیه فقط وجود چنین شبکه‌ای را تضمین می‌کند و درباره تعداد نوروهای لازم یا چگونگی یافتن وزن‌ها چیزی نمی‌گوید. در عمل، شبکه‌های عمیق‌تر (با لایه‌های بیشتر) معمولاً با تعداد پارامتر کمتری به دقت مشابه می‌رسند و به همین دلیل در کاربردها ترجیح داده می‌شوند.

<sup>۸</sup>Cybenko

## ۴-۲ آموزش شبکه‌های عمیق

آموزش شبکه عصبی یعنی یافتن وزن‌ها و بایاس‌هایی که خروجی شبکه را به خروجی مطلوب نزدیک کند. برای این کار ابتدا یک تابع هزینه<sup>۹</sup> تعریف می‌شود که فاصله پیش‌بینی شبکه از مقادیر واقعی را اندازه می‌گیرد. پرکاربردترین تابع هزینه در مسائل رگرسیون، میانگین مربعات خطا<sup>۱۰</sup> است:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (u_{\theta}(\mathbf{x}_i) - u_i)^2, \quad (۷-۲)$$

که در آن  $u_{\theta}$  خروجی شبکه با پارامترهای  $\theta$  و  $u_i$  مقدار واقعی نظیر ورودی  $\mathbf{x}_i$  است. کمینه‌سازی تابع هزینه معمولاً با روش‌های مبتنی بر گرادیان انجام می‌شود. ساده‌ترین آن‌ها، روش گرادیان کاهشی است که در هر گام، پارامترها را در خلاف جهت گرادیان تابع هزینه به‌روز می‌کند:

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} \mathcal{L}(\theta_k), \quad (۸-۲)$$

که  $\eta$  نرخ یادگیری<sup>۱۱</sup> و  $\mathcal{L}$  تابع هزینه است. محاسبه کارای گرادیان در شبکه‌های بزرگ با الگوریتم پس‌انتشار<sup>۱۲</sup> انجام می‌شود که در واقع کاربرد قاعده زنجیره‌ای مشتق روی گراف محاسباتی شبکه است که توسط رومل‌هارت و همکارانش معرفی و فراگیر شد [۱۰]. دو بهینه‌ساز پرکاربرد که در این پایان‌نامه نیز از آن‌ها استفاده شده عبارت‌اند از: **روش آدام**. آدام<sup>۱۳</sup> یکی از محبوب‌ترین بهینه‌سازها در یادگیری عمیق است که برای هر پارامتر، نرخ یادگیری تطبیقی جداگانه‌ای نگه می‌دارد [۱۲]. قاعده به‌روزرسانی آن با استفاده از میانگین‌های متحرک گرادیان ( $m$ ) و مربع گرادیان ( $v$ ) چنین است:

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1 - \beta_1) g_k, \\ v_k &= \beta_2 v_{k-1} + (1 - \beta_2) g_k^2, \\ \theta_{k+1} &= \theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}, \end{aligned} \quad (۹-۲)$$

که در آن  $g_k$  گرادیان در گام  $k$ ام است و  $\hat{m}_k$  و  $\hat{v}_k$  نسخه‌های اصلاح بایاس‌شده  $m_k$  و  $v_k$

<sup>۹</sup>Loss Function

<sup>۱۰</sup>Mean Squared Error (MSE)

<sup>۱۱</sup>Learning Rate

<sup>۱۲</sup>Backpropagation

<sup>۱۳</sup>Adaptive Moment Estimation (Adam)

هستند. مقادیر پیش فرض  $\beta_1 = 0.9$ ،  $\beta_2 = 0.999$  و  $\epsilon = 10^{-8}$  در بیشتر کاربردها به خوبی عمل می‌کنند.

**روش ال-بی‌اف‌جی‌اس.** این روش از خانواده روش‌های شبه‌نیوتنی است که با تقریب معکوس ماتریس هشین، از اطلاعات انحنای تابع هزینه نیز بهره می‌برد و معمولاً در نزدیکی نقطه بهینه همگرایی سریعی دارد [۱۳]. نسخه حافظه‌محدود آن (L-BFGS) برای مسائل با پارامترهای زیاد مناسب است. در آموزش شبکه‌های آگاه از فیزیک، راهبرد رایج این است که ابتدا چند هزار گام با آدام پیش می‌روند و سپس برای دقت نهایی، آموزش را با L-BFGS ادامه می‌دهند؛ ما نیز در پیاده‌سازی خود همین راهبرد را به کار برده‌ایم.

نکته مهم دیگر، مقداردهی اولیه وزن‌هاست. اگر وزن‌ها نامناسب مقداردهی شوند، آموزش یا بسیار کند می‌شود یا اصلاً همگرا نمی‌شود. روش گزایه<sup>۱۴</sup> که واریانس وزن‌های اولیه را متناسب با تعداد ورودی و خروجی هر لایه تنظیم می‌کند [۱۴]، انتخابی استاندارد برای شبکه‌هایی با فعال‌ساز تانژانت هیپربولیک است و در این پژوهش نیز از آن استفاده شده است.

## ۵-۲ مشتق‌گیری خودکار

مشتق‌گیری خودکار مجموعه‌ای از تکنیک‌ها برای محاسبه دقیق مشتق توابعی است که با یک برنامه کامپیوتری پیاده‌سازی شده‌اند [۱۵]. تفاوت آن با مشتق‌گیری عددی (مانند تفاضل محدود) در این است که خطای برشی ندارد و نتیجه تا دقت ماشین دقیق است؛ و تفاوت آن با مشتق‌گیری نمادی در این است که با عبارت‌های ریاضی سروکار ندارد، بلکه روی گراف محاسباتی برنامه و با اعمال قاعده زنجیره‌ای روی عملیات پایه عمل می‌کند، پس با مشکل انفجار عبارت‌ها هم مواجه نیست.

دو حالت اصلی مشتق‌گیری خودکار عبارت‌اند از حالت مستقیم<sup>۱۵</sup> و حالت معکوس<sup>۱۶</sup>. در حالت معکوس که همان ایده حاکم بر الگوریتم پس‌انتشار است، ابتدا برنامه به ترتیب اجرا و مقادیر میانی ذخیره می‌شوند و سپس مشتق‌ها از خروجی به سمت ورودی منتشر می‌گردند. مزیت بزرگ این حالت آن است که گرادیان یک خروجی اسکالر نسبت به هزاران ورودی (مثلاً وزن‌های شبکه) تنها با حدود دو برابر هزینه یک اجرای مستقیم به دست می‌آید؛ به همین دلیل ستون فقرات آموزش شبکه‌های عمیق محسوب می‌شود.

اهمیت مشتق‌گیری خودکار در این پایان‌نامه دو جنبه دارد: اولاً برای به‌روزرسانی

<sup>14</sup>Xavier Initialization

<sup>15</sup>Forward Mode

<sup>16</sup>Reverse Mode

وزن‌های شبکه در فرایند آموزش به آن نیاز داریم (همان کاربرد معمول) و ثانیاً، و مهم‌تر، برای محاسبه مشتق‌های جواب نسبت به متغیرهای ورودی شبکه یعنی مکان و زمان. همان‌طور که در فصل سوم خواهیم دید، همین قابلیت است که اجازه می‌دهد باقیمانده معادله دیفرانسیل را مستقیماً از روی شبکه بسازیم، بدون آن‌که نیازی به شبکه محاسباتی یا فرمول تفاضل محدود باشد. کتابخانه‌های مدرن یادگیری عمیق مانند تنسورفلو [۲۳] و جکس [۲۴] این قابلیت را به‌صورت توکار فراهم می‌کنند.

## ۶-۲ پیشینه پژوهش

ایده استفاده از شبکه‌های عصبی برای حل معادلات دیفرانسیل قدمتی بیش از سه دهه دارد. لاگاریس و همکارانش در سال ۱۹۹۸ نشان دادند که می‌توان جواب معادلات دیفرانسیل معمولی و جزئی را با یک شبکه عصبی تقریب زد، به‌گونه‌ای که شرایط مرزی به‌صورت ساختاری در پاسخ پیشنهادی لحاظ شود و باقیمانده معادله در تعدادی نقطه نمونه کمینه گردد [۲]. هرچند این کار در زمان خود مورد توجه قرار گرفت، اما محدودیت توان محاسباتی و نبود ابزارهای مشتق‌گیری خودکار کارآمد باعث شد این مسیر سال‌ها کم‌رونق بماند.

موج دوم این پژوهش‌ها با ورود فرایندهای گاوسی<sup>۱۷</sup> به این حوزه شکل گرفت. رئیسی و همکارانش نشان دادند که با طراحی کرنل‌های سازگار با عملگر دیفرانسیل می‌توان جواب معادلات خطی [۳] و سپس با خطی‌سازی موضعی، معادلات غیرخطی [۴] را استنتاج کرد؛ روشی که علاوه بر جواب، برآورد عدم قطعیت هم ارائه می‌داد. اما ماهیت بیزی این روش‌ها و نیاز به خطی‌سازی، کاربرد آن‌ها را در مسائل به‌شدت غیرخطی محدود می‌کرد.

در مسیری موازی، کارپاتنه و همکارانش چارچوب «علم داده هدایت‌شده با نظریه» را مطرح کردند که هدف آن سازگار کردن مدل‌های یادگیری ماشین با دانش علمی موجود در حوزه مسئله است [۵]. همچنین پژوهش‌های اشمیت و لیپسون [۶] و روش SINDy معرفی‌شده توسط برانتون و همکارانش [۷] نشان دادند که می‌توان قوانین حاکم بر یک سیستم دینامیکی را مستقیماً از روی داده کشف کرد؛ هرچند این روش‌ها معمولاً به کتابخانه‌ای از جملات کاندید نیاز دارند و برای معادلات جزئی پیچیده چندان مقیاس‌پذیر نیستند.

در این میان، مقاله رئیسی، پردیکاریس و کارنیاداکیس در سال ۲۰۱۹ نقطه عطفی محسوب می‌شود [۱]. آن‌ها با ترکیب شبکه‌های عصبی عمیق و مشتق‌گیری خودکار،

<sup>17</sup>Gaussian Process



چارچوبی یکپارچه برای هر دو دسته مسائل مستقیم (یافتن جواب) و معکوس (کشف پارامترها) ارائه کردند که نیازی به خطی‌سازی، شبکه محاسباتی یا کتابخانه جملات کاندید ندارد. سادگی ایده، عمومیت آن و نتایج قانع‌کننده روی مسائل کلاسیک (از معادله برگرز و شرودینگر تا ناویر - استوکس) باعث شد این مقاله به سرعت به یکی از آثار مرجع حوزه تبدیل شود و صدها پژوهش بعدی را رقم بزند؛ به گونه‌ای که امروزه مرورهای جامعی مانند [۸] صرفاً به دسته‌بندی و تحلیل این خانواده از روش‌ها اختصاص یافته است. این پایان‌نامه نیز بر مبنای همین مقاله مرجع بنا شده و می‌کوشد ایده‌های آن را در سطح کارشناسی، هم به صورت نظری و هم به صورت عملی، بازتولید و ارزیابی کند.

## ۷-۲ جمع‌بندی فصل

در این فصل، مفاهیم پایه مورد نیاز این پژوهش مرور شد: معادلات دیفرانسیل جزئی و مثال‌های مهم آن‌ها، روش‌های عددی کلاسیک و محدودیت‌هایشان، ساختار و آموزش شبکه‌های عصبی، مشتق‌گیری خودکار و پیشینه پژوهش در زمینه یادگیری ماشین آگاه از فیزیک. در فصل بعد، با تکیه بر همین مبانی، چارچوب شبکه‌های عصبی آگاه از فیزیک به تفصیل تشریح می‌شود.

## فصل ۳

### شبکه‌های عصبی آگاه از فیزیک

در این فصل، چارچوب شبکه‌های عصبی آگاه از فیزیک که روش اصلی این پژوهش است، به تفصیل تشریح می‌شود. ابتدا صورت کلی مسئله و ایده محوری روش بیان می‌گردد و سپس دو گونه مدل زمان‌پیوسته و زمان‌گسسته برای مسائل مستقیم معرفی می‌شوند. پس از آن، فرمول‌بندی مسئله معکوس و کشف پارامترها توضیح داده می‌شود و در پایان، نکات عملی مهم در پیاده‌سازی و آموزش این شبکه‌ها مرور می‌گردد. مبنای این فصل، مقاله رئیسی و همکارانش [۱] است.

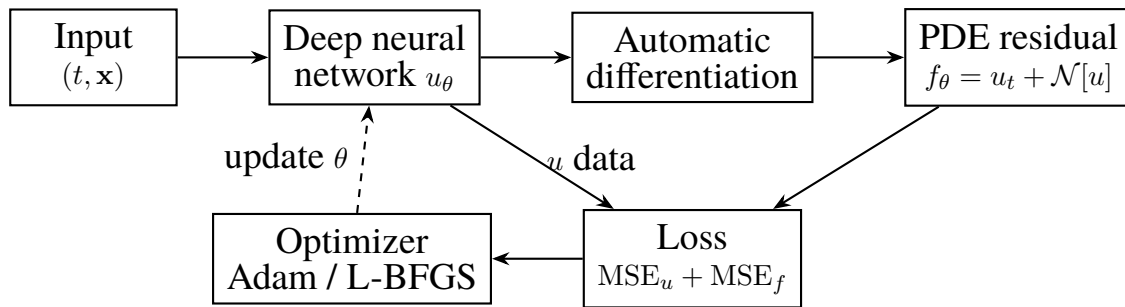
#### ۱-۳ صورت کلی مسئله و ایده محوری

معادلات دیفرانسیل جزئی پارامتری و غیرخطی به شکل کلی زیر را در نظر بگیرید:

$$u_t + \mathcal{N}[u; \lambda] = 0, \quad \mathbf{x} \in \Omega, \quad t \in [0, T], \quad (1-3)$$

که در آن  $u(t, \mathbf{x})$  جواب مجهول،  $\mathcal{N}[\cdot; \lambda]$  یک عملگر دیفرانسیل غیرخطی با پارامترهای  $\lambda$  و  $\Omega$  دامنه مکانی مسئله است. این صورت کلی، طیف وسیعی از مسائل از قوانین بقا و فرایندهای پخش تا سامانه‌های واکنش - انتشار را در بر می‌گیرد. برای نمونه، معادله برگرز (۳-۲) متناظر با حالت  $\mathcal{N}[u; \lambda] = \lambda_1 u u_x - \lambda_2 u_{xx}$  با  $\lambda = (\lambda_1, \lambda_2)$  است. با فرض در دست بودن تعدادی مشاهده (احتمالاً نویزی) از سیستم، دو مسئله مجزا مطرح می‌شود. مسئله نخست، مسئله مستقیم است: با فرض معلوم بودن پارامترهای  $\lambda$ ، درباره حالت پنهان  $u(t, \mathbf{x})$  چه می‌توان گفت؟ مسئله دوم، مسئله معکوس یا کشف معادله است: کدام پارامترهای  $\lambda$  داده‌های مشاهده‌شده را به بهترین شکل توصیف می‌کنند؟

ایده محوری شبکه‌های آگاه از فیزیک آن است که جواب  $u(t, \mathbf{x})$  را با یک شبکه عصبی عمیق  $u_\theta(t, \mathbf{x})$  با پارامترهای  $\theta$  تقریب بزنیم و سپس باقیمانده معادله را به صورت



شکل ۳-۱: نمای کلی چارچوب شبکه عصبی آگاه از فیزیک: شبکه جواب، مشتق‌گیری خودکار، شبکه باقیمانده و حلقه آموزش

زیر تعریف کنیم:

$$f(t, \mathbf{x}) := u_t + \mathcal{N}[u], \quad (۲-۳)$$

که در آن  $u$  همان خروجی شبکه است. نکته کلیدی اینجا است که چون مشتق‌های  $u_t$ ،  $u_{xx}$  و غیره همگی با مشتق‌گیری خودکار از روی شبکه به دست می‌آیند، خود  $f$  نیز یک شبکه عصبی (با همان پارامترهای  $\theta$  ولی توابع فعال‌ساز متفاوت) محسوب می‌شود؛ به این شبکه، شبکه آگاه از فیزیک گفته می‌شود. اگر شبکه  $u$  به خوبی آموزش ببیند، باید در همه نقاط دامنه  $f \approx 0$  باشد، یعنی معادله دیفرانسیل ارضا شود. بنابراین کافی است تابع هزینه‌ای طراحی کنیم که هم فاصله از داده‌ها و هم باقیمانده معادله را همزمان کمینه کند. نمایی کلی از این چارچوب در شکل ۳-۱ نشان داده شده است.

### ۲-۳ مدل‌های زمان‌پیوسته برای مسئله مستقیم

در مدل زمان‌پیوسته، شبکه ورودی  $(t, \mathbf{x})$  را می‌گیرد و مستقیماً  $u(t, \mathbf{x})$  را پیش‌بینی می‌کند؛ به همین دلیل، این مدل‌ها خانواده جدیدی از تقریب‌زننده‌های تابعی زمانی - مکانی به شمار می‌روند. پارامترهای مشترک شبکه‌های  $u$  و  $f$  با کمینه‌سازی تابع هزینه زیر آموخته می‌شوند:

$$\text{MSE} = \text{MSE}_u + \text{MSE}_f, \quad (۳-۳)$$

که در آن:

$$\begin{aligned} \text{MSE}_u &= \frac{1}{N_u} \sum_{i=1}^{N_u} |u(t_u^i, \mathbf{x}_u^i) - u^i|^2, \\ \text{MSE}_f &= \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, \mathbf{x}_f^i)|^2. \end{aligned} \quad (۴-۳)$$

در اینجا  $\{t_u^i, \mathbf{x}_u^i, u^i\}_{i=1}^{N_u}$  داده‌های آموزشی روی شرط اولیه و مرز هستند و  $\{t_f^i, \mathbf{x}_f^i\}_{i=1}^{N_f}$  نقاط هم‌مکانی<sup>۱</sup> نامیده می‌شوند؛ یعنی نقاطی از دامنه که در آن‌ها صرفاً ارضای معادله (صفر بودن  $f$ ) اجبار می‌شود، بدون آن‌که مقدار جواب در آن نقاط معلوم باشد. نکته قابل‌توجه، کارایی داده‌ای این روش است: در مثال‌های مقاله مرجع، تنها با چند ده یا چند صد نقطه داده اولیه و مرزی، جواب کل دامنه با دقت بالا بازسازی می‌شود، زیرا بخش  $\text{MSE}_f$  مانند یک منظم‌ساز فیزیکی عمل می‌کند و فضای جواب‌های قابل‌قبول را به شدت محدود می‌سازد. در مسئله نمونه این پایان‌نامه (معادله برگرز)، شبکه  $f(t, x)$  شکل صریح زیر را دارد:

$$f := u_t + uu_x - \nu u_{xx}, \quad (۵-۳)$$

و تابع هزینه دقیقاً همان (۳-۳) است. یکی از محدودیت‌های مدل زمان‌پیوسته آن است که وقتی تعداد نقاط هم‌مکانی لازم زیاد می‌شود (مثلاً در مسائل با دینامیک تند یا ابعاد بالا)، هزینه محاسباتی آموزش افزایش می‌یابد. برای غلبه بر این مشکل، گونه دوم مدل‌ها معرفی شده است که در بخش بعد می‌آید.

### ۳-۳ مدل‌های زمان‌گسسته

در مدل زمان‌گسسته، به جای آن‌که شبکه کل بازه زمانی را یکجا بیاموزد، از یک طرح گام‌زنی زمانی ضمنی از خانواده رانگ - کوتا<sup>۲</sup> با  $q$  مرحله استفاده می‌شود. با اعمال

<sup>۱</sup>Collocation Points

<sup>۲</sup>Runge-Kutta

صورت کلی روش‌های رانگ - کوتا به معادله (۳-۱) داریم:

$$\begin{aligned} u^{n+c_i} &= u^n - \Delta t \sum_{j=1}^q a_{ij} \mathcal{N}[u^{n+c_j}], \quad i = 1, \dots, q, \\ u^{n+1} &= u^n - \Delta t \sum_{j=1}^q b_j \mathcal{N}[u^{n+c_j}], \end{aligned} \quad (۳-۶)$$

که در آن  $\{a_{ij}, b_j, c_j\}$  ضرایب طرح رانگ - کوتا هستند. در اینجا به جای یک شبکه،  $q+1$  خروجی شبکه در نظر گرفته می‌شود که مراحل میانی  $u^{n+c_1}, \dots, u^{n+c_q}$  و جواب گام بعدی  $u^{n+1}$  را پیش‌بینی می‌کنند و تابع هزینه، سازگاری این خروجی‌ها با روابط (۳-۶) و داده‌های دو لحظه زمانی  $t^n$  و  $t^{n+1}$  را اجبار می‌کند.

مزیت اصلی این رهیافت آن است که چون طرح رانگ - کوتای ضمنی ذاتاً پایدار است، می‌توان گام‌های زمانی بسیار بزرگ برداشت و حتی با تعداد مراحل زیاد (مثلاً  $q = 500$ ) در برخی مثال‌های مقاله) به دقت زمانی بالایی رسید؛ کاری که در روش‌های کلاسیک به سبب هزینه حل دستگاه‌های غیرخطی بزرگ عملاً ناممکن است. به بیان دیگر، شبکه عصبی نقش حل‌کننده دستگاه غیرخطی ناشی از گسسته‌سازی ضمنی را بازی می‌کند. شرح کامل نظریه و انواع طرح‌های رانگ - کوتا را می‌توان در منابع استاندارد روش‌های عددی معادلات دیفرانسیل معمولی یافت [۲۲]. در این پایان‌نامه تمرکز پیاده‌سازی بر مدل زمان‌پیوسته است، اما آشنایی با مدل زمان‌گسسته برای درک کامل چارچوب ضروری بود.

### ۳-۴ مسئله معکوس و کشف معادله

در مسئله معکوس، علاوه بر جواب  $u$ ، پارامترهای  $\lambda$  عملگر دیفرانسیل نیز مجهول‌اند و باید از روی داده‌ها فراگرفته شوند. خوشبختانه در چارچوب آگاه از فیزیک، این کار به‌طور طبیعی انجام می‌شود: کافی است  $\lambda$  را نیز جزء پارامترهای قابل‌آموزش شبکه به حساب آوریم و همان تابع هزینه (۳-۳) را نسبت به  $(\theta, \lambda)$  کمینه کنیم. در اینجا داده‌های آموزشی، مشاهده‌های پراکنده جواب در سراسر دامنه زمانی - مکانی هستند (نه فقط روی شرط اولیه و مرز) و نقاط هم‌مکانی نیز می‌توانند همان نقاط داده باشند. برای مثال، در مسئله کشف معادله برگرز، باقیمانده به شکل زیر در نظر گرفته می‌شود:

$$f := u_t + \lambda_1 u u_x - \lambda_2 u_{xx}, \quad (۳-۷)$$

که  $\lambda_1$  و  $\lambda_2$  از مقدار اولیه دلخواه (مثلاً صفر) شروع می‌کنند و در جریان آموزش به مقادیر واقعی خود ( $\lambda_1 = 1$  و  $\lambda_2 = 0.01/\pi$ ) همگرا می‌شوند. نکته جالب این است که شبکه در این حالت هم‌زمان دو کار انجام می‌دهد: هم جواب را در کل دامنه بازسازی می‌کند و هم پارامترهای معادله حاکم را شناسایی می‌نماید. مقاله مرجع نشان می‌دهد که این شناسایی حتی در حضور نویز در داده‌ها نیز با دقت قابل‌قبولی انجام می‌شود؛ موضوعی که در فصل چهارم، در بخش عملی این پایان‌نامه نیز آزموده خواهد شد.

تفاوت مهم مسئله معکوس با روش‌هایی مانند SINDy (که در فصل دوم معرفی شد) در این است که در اینجا نیازی به کتابخانه‌ای از جملات کاندید نیست و مشتق‌های لازم نیز به‌جای تفاضل محدود روی داده‌های نویزی، با مشتق‌گیری خودکار از روی شبکه‌ای هموار محاسبه می‌شوند؛ همین امر روش را در برابر نویز مقاوم‌تر می‌سازد.

### ۳-۵ نکات عملی در پیاده‌سازی و آموزش

تجربه مقاله مرجع و پژوهش‌های بعدی نشان می‌دهد که موفقیت آموزش شبکه‌های آگاه از فیزیک به چند انتخاب مهم بستگی دارد که در ادامه خلاصه می‌شوند.

**معماری شبکه.** در بیشتر مثال‌ها از پرسپترون‌های چندلایه نسبتاً کوچک (مثلاً ۴ تا ۸ لایه پنهان با ۲۰ تا ۵۰ نورون در هر لایه) با فعال‌ساز تانژانت هیپربولیک استفاده می‌شود و معمولاً نیازی به منظم‌سازهای اضافی مانند افت تصادفی<sup>۳</sup> نیست، زیرا خود جمله فیزیکی تابع هزینه نقش منظم‌ساز را ایفا می‌کند.

**بهینه‌ساز.** راهبرد استاندارد، شروع آموزش با چند هزار گام آدام و سپس ادامه آن با L-BFGS تا همگرایی است. آدام در ابتدا سریع پیش می‌رود و از نقاط زینی عبور می‌کند، در حالی که L-BFGS در نزدیکی بهینه، دقت نهایی را به‌طور چشمگیری بهبود می‌بخشد. در این پایان‌نامه نیز دقیقاً همین راهبرد دو مرحله‌ای به کار رفته است.

**نقاط هم‌مکانی.** تعداد و توزیع نقاط هم‌مکانی اثر مستقیمی بر دقت و هزینه آموزش دارد. معمولاً چند هزار نقطه که به‌صورت تصادفی یکنواخت (یا با نمونه‌برداری لاتین هایپر مکعب<sup>۴</sup>) از دامنه انتخاب می‌شوند کفایت می‌کند. در مسائلی که جواب در ناحیه خاصی تغییرات تندی دارد، تمرکز بیشتر نقاط در آن ناحیه می‌تواند مفید باشد.

**مقیاس‌بندی.** از آنجا که ورودی‌های شبکه (زمان و مکان) و خروجی آن ممکن است در بازه‌های متفاوتی باشند، نرمال‌سازی ورودی‌ها به بازه‌ای مانند  $[-1, 1]$  معمولاً به همگرایی سریع‌تر و پایدارتر کمک می‌کند. در پیاده‌سازی این پایان‌نامه، دامنه مسئله

<sup>3</sup>Dropout

<sup>4</sup>Latin Hypercube Sampling

جدول ۳-۱: مقایسه مدل‌های زمان‌پیوسته و زمان‌گسسته در چارچوب آگاه از فیزیک

| ویژگی                    | مدل زمان‌پیوسته  | مدل زمان‌گسسته                    |
|--------------------------|------------------|-----------------------------------|
| ورودی شبکه               | $(t, x)$         | $x$ در یک لحظه زمانی              |
| خروجی شبکه               | $u(t, x)$        | مراحل رانگ - کوتا و $u^{n+1}$     |
| داده آموزشی              | شرط اولیه و مرزی | جواب در دو لحظه $t^n$ و $t^{n+1}$ |
| مزیت اصلی                | سادگی و یکپارچگی | گام زمانی بزرگ و پایدار           |
| کاربرد در این پایان‌نامه | پیاده‌سازی شده   | فقط بررسی نظری                    |

چنان انتخاب شده که نیازی به مقیاس‌بندی اضافی نباشد.  
در جدول ۳-۱ دو گونه مدل معرفی شده در این فصل از نظر ویژگی‌های اصلی مقایسه شده‌اند.

### ۳-۶ جمع‌بندی فصل

در این فصل، چارچوب شبکه‌های عصبی آگاه از فیزیک تشریح شد: ایده تعریف شبکه باقیمانده با مشتق‌گیری خودکار، تابع هزینه ترکیبی داده و فیزیک، مدل‌های زمان‌پیوسته و زمان‌گسسته برای مسئله مستقیم، و فرمول‌بندی مسئله معکوس برای کشف پارامترها. در فصل بعد، مدل زمان‌پیوسته برای معادله برگرز پیاده‌سازی و نتایج عددی آن گزارش و تحلیل می‌شود.

## فصل ۴

### پیاده‌سازی و ارزیابی نتایج

در این فصل، بخش عملی پایان‌نامه تشریح می‌شود. پس از معرفی محیط و ابزار پیاده‌سازی، حل‌گر مرجعی که برای تولید جواب دقیق معادله برگرز توسعه داده شده معرفی و راستی‌آزمایی می‌گردد. سپس نتایج حل مستقیم معادله با مدل زمان‌پیوسته و نتایج کشف پارامترها در مسئله معکوس گزارش می‌شود و در پایان، یافته‌ها تحلیل و با نتایج مقاله مرجع مقایسه می‌گردند.

#### ۱-۴ محیط و ابزار پیاده‌سازی

همه کدهای این پژوهش با زبان پایتون نسخه 3.11 نوشته شده‌اند. برای پیاده‌سازی شبکه عصبی و مشتق‌گیری خودکار از کتابخانه جکس<sup>۱</sup> استفاده شده است [۲۴] که امکان ترکیب تبدیل‌های تابعی مانند گرادین‌گیری، برداری‌سازی و کامپایل به‌موقع را فراهم می‌کند و برای پژوهش‌های محاسبات علمی گزینه‌ای سبک و مناسب است. محاسبات عددی کمکی با نامپای و سای‌پای و ترسیم نمودارها با مت‌پلات‌لیب انجام شده است. بهینه‌سازی دومرحله‌ای شامل پیاده‌سازی دستی الگوریتم آدام و فراخوانی پیاده‌سازی L-BFGS-B از کتابخانه سای‌پای است.

آموزش‌ها روی پردازنده مرکزی (بدون کارت گرافیک) اجرا شده‌اند تا شرایط بازتولید برای خواننده با امکانات معمولی نیز فراهم باشد. برای تکرارپذیری نتایج، بذر همه مولدهای اعداد تصادفی ثابت نگه داشته شده و نسخه دقیق کد اجرا شده به‌همراه راهنمای نصب و اجرا در پیوست الف و پوشه code ارائه شده است.

#### ۲-۴ حل‌گر مرجع و راستی‌آزمایی آن

برای آن‌که دقت شبکه آگاه از فیزیک به‌صورت مستقل سنجیده شود، یک حل‌گر مرجع برای معادله برگرز پیاده‌سازی شد که هیچ ارتباطی با شبکه عصبی ندارد. در این حل‌گر،

---

<sup>1</sup>JAX



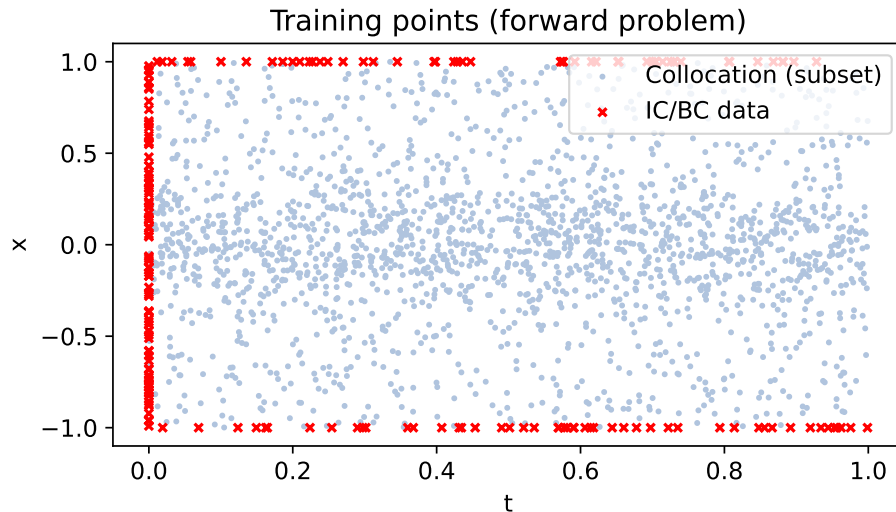
گسسته‌سازی مکانی با روش طیفی فوریه روی  $10^{24}$  مود و با قانون پالایش دو - سوم برای حذف اثر هم‌پوشانی انجام می‌شود [۲۰] و گام‌زنی زمانی با روش ETDRK4 (رانگ - کوتای مرتبه چهار نمایی) با گام  $dt = 0.001$  صورت می‌گیرد که برای جمله پخش سفت، پایداری مناسبی دارد [۲۱]. خروجی این حل‌گر روی شبکه  $101 \times 1024$  زمانی - مکانی ذخیره و به‌عنوان جواب دقیق در ارزیابی‌ها استفاده شده است.

برای اطمینان از صحت حل‌گر مرجع، دو آزمون مستقل انجام شد. نخست، آزمون همگرایی: با نصف کردن گام زمانی، تغییر نسبی جواب فقط  $6.2 \times 10^{-9}$  و با دو برابر کردن تفکیک مکانی، تغییر نسبی  $4.1 \times 10^{-4}$  به دست آمد که رفتار مورد انتظار یک روش مرتبه چهار زمانی و طیفی مکانی است. دوم، مقایسه با جواب نیمه‌تحلیلی کول - هوپف [۱۸] که با انتگرال‌گیری عددی مستقیم (بدون هیچ گام‌زنی زمانی) محاسبه شد: بیشترین اختلاف در ۱۹ نقطه نمونه در لحظه  $t = 1$  کمتر از  $6 \times 10^{-4}$  بود و در نقاط دور از جبهه موج به مرتبه  $10^{-8}$  می‌رسید. این دو آزمون نشان می‌دهد که جواب مرجع با دقتی بسیار بالاتر از دقت مورد انتظار شبکه عصبی محاسبه شده و مبنای قابل‌اعتمادی برای مقایسه است. گفتنی است که برای معادله برگرز، جواب دقیق تحلیلی نیز به‌صورت سری فوریه در مراجع کلاسیک موجود است [۱۹].

### ۳-۴ حل مستقیم معادله برگرز

مسئله مستقیم به این صورت تعریف شد: با معلوم فرض کردن ضرایب معادله برگرز  $(\lambda_1 = 1 \text{ و } \lambda_2 = 0.01/\pi)$ ، جواب  $u(t, x)$  در دامنه  $x \in [-1, 1]$  و  $t \in [0, 1]$  با شرط اولیه  $u(0, x) = -\sin(\pi x)$  و شرایط مرزی  $u(t, -1) = u(t, 1) = 0$  به دست آید. شبکه مورد استفاده، یک پرسپترون چندلایه با ۴ لایه پنهان و ۵۰ نورون در هر لایه با فعال‌ساز تانژانت هیپربولیک است که در مجموع ۷۸۵۱ پارامتر دارد. داده آموزشی شامل ۲۰۰ نقطه روی شرط اولیه (۱۰۰ نقطه) و مرزها (۱۰۰ نقطه) است و ۱۰۰۰۰ نقطه هم‌مکانی نیز در نظر گرفته شد که نیمی از آن‌ها به‌صورت یکنواخت از کل دامنه و نیم دیگر به‌صورت متمرکز حول  $x = 0$  (محل تشکیل جبهه تند موج) نمونه‌برداری شدند. توزیع نقاط آموزشی در شکل ۱-۴ نشان داده شده است. خلاصه تنظیمات آموزش در جدول ۱-۴ آمده است.

آموزش در دو مرحله انجام شد: ابتدا ۶۰۰۰ گام با بهینه‌ساز آدام و نرخ یادگیری  $10^{-3}$  و سپس ادامه آموزش با روش L-BFGS تا سقف ۶۰۰۰ ارزیابی تابع. مقدار تابع هزینه در پایان مرحله آدام  $5.85 \times 10^{-3}$  و در پایان مرحله L-BFGS  $8.71 \times 10^{-5}$  به دست آمد که سهم جملات داده و فیزیک در آن به‌ترتیب  $2.35 \times 10^{-5}$  و  $6.36 \times 10^{-5}$  بود. نمودار تاریخچه آموزش در شکل ۲-۴ رسم شده است که کاهش سریع خطا در مرحله آدام و



شکل ۴-۱: توزیع نقاط آموزشی در مسئله مستقیم: نقاط داده روی شرط اولیه و مرزها و نقاط هم‌مکانی در دامنه

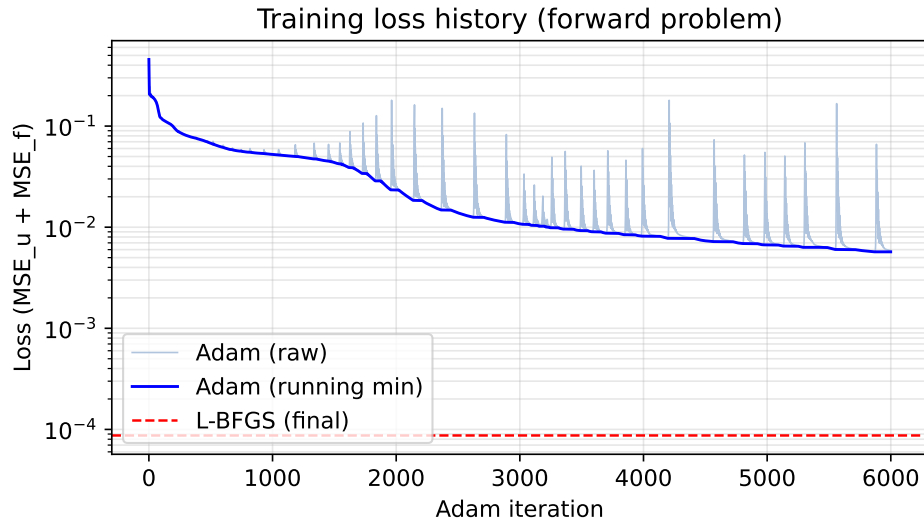
جدول ۴-۱: تنظیمات آموزش مدل زمان‌پیوسته در مسئله مستقیم

| مشخصات                                                        | مؤلفه                   |
|---------------------------------------------------------------|-------------------------|
| ۴ لایه پنهان، ۵۰ نورون در هر لایه، فعال‌ساز تانژانت هیپربولیک | معماری شبکه             |
| ۷۸۵۱                                                          | تعداد پارامترها         |
| ۲۰۰ نقطه (۱۰۰ شرط اولیه و ۱۰۰ شرط مرزی)                       | نقاط داده ( $N_u$ )     |
| ۱۰۰۰۰ نقطه (نیمی یکنواخت و نیمی متمرکز حول جبهه)              | نقاط هم‌مکانی ( $N_f$ ) |
| روش گزاولیه                                                   | مقداردهی اولیه          |
| ۶۰۰۰ گام آدام و سپس L-BFGS                                    | بهینه‌سازی              |
| ممیز شناور ۶۴ بیتی                                            | دقت محاسبات             |

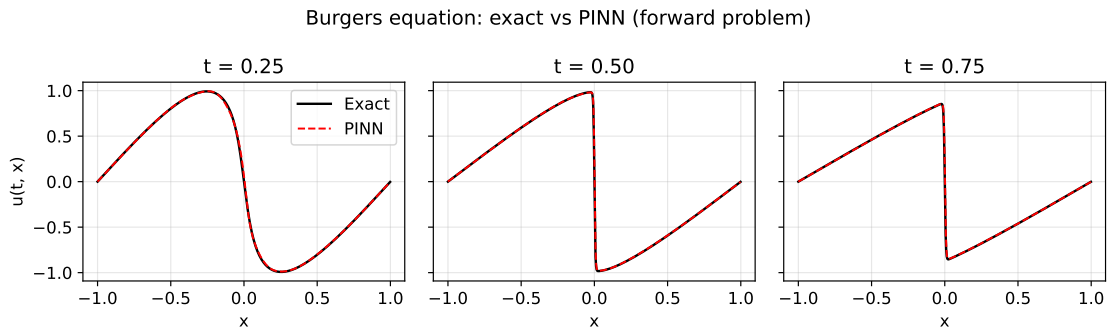
بهبود نهایی در مرحله L-BFGS را نشان می‌دهد.

برای سنجش دقت، خروجی شبکه روی کل شبکه مرجع  $101 \times 1024$  ارزیابی و با جواب دقیق مقایسه شد. خطای نسبی در نرم  $L_2$  برابر  $2.15 \times 10^{-2}$  به دست آمد. مقایسه جواب دقیق و پیش‌بینی شبکه در سه لحظه زمانی  $t = 0.25$ ،  $0.5$  و  $0.75$  در شکل ۴-۳ نشان داده شده است. همان‌طور که دیده می‌شود، شبکه شکل کلی جواب و محل جبهه موج را به خوبی بازسازی کرده است. نقشه خطای مطلق در شکل ۴-۴ نیز نشان می‌دهد که خطا تقریباً در همه دامنه ناچیز است و فقط در لایه بسیار باریک جبهه موج (حوالی  $x = 0$  برای  $t > 0.3$ ) متمرکز شده است؛ ناحیه‌ای که گرادیان جواب در آن بسیار تند است و حتی برای روش‌های عددی کلاسیک نیز دشوارترین بخش مسئله محسوب می‌شود.

در جدول ۴-۲ نتایج این پایان‌نامه با نتایج گزارش‌شده در مقاله مرجع [۱] مقایسه شده است. همان‌طور که مشاهده می‌شود، دقت به دست آمده در این بازتولید کمتر



شکل ۴-۲: نمودار تاریخچه تابع هزینه در مسئله مستقیم: کاهش سریع در مرحله آدام و بهبود نهایی با روش شبه نیوتنی

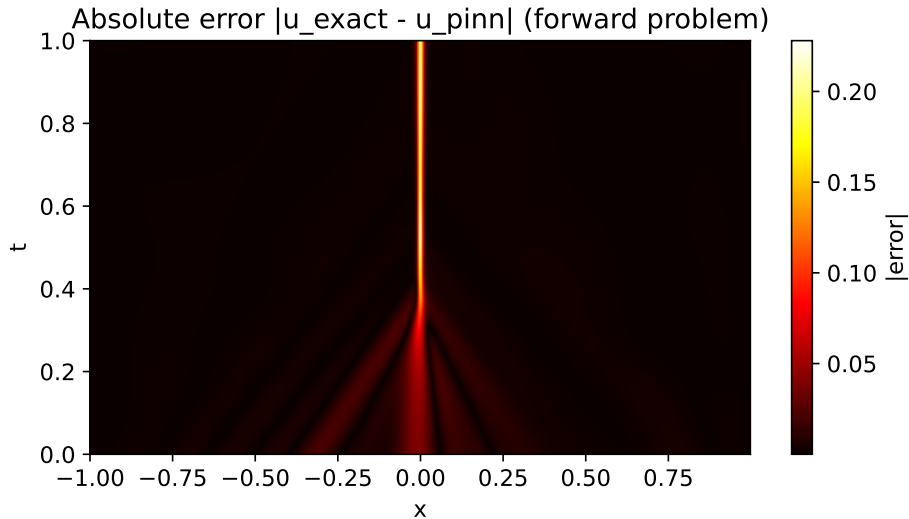


شکل ۴-۳: مقایسه جواب دقیق و پیش‌بینی شبکه آگاه از فیزیک در سه لحظه زمانی در مسئله مستقیم معادله برگرز

از عدد گزارش شده در مقاله است که با توجه به تفاوت امکانات (آموزش روی پردازنده مرکزی در برابر کارت گرافیک، و بودجه محدودتر تکرارهای بهینه‌سازی) و تفاوت معماری شبکه، امری مورد انتظار است. نکته مهم، بازتولید کیفی رفتار روش است: دستیابی به جوابی قابل قبول در کل دامنه، تنها با ۲۰۰ نقطه داده اولیه و مرزی.

## ۴-۴ کشف پارامترهای معادله برگرز

در مسئله معکوس، ضرایب  $\lambda_1$  و  $\lambda_2$  در باقیمانده  $f = u_t + \lambda_1 u u_x - \lambda_2 u_{xx}$  مجهول فرض شدند و هدف، شناسایی آن‌ها از روی مشاهده‌های پراکنده جواب بود. داده آموزشی شامل ۵۰۰۰ نقطه پراکنده در دامنه زمانی - مکانی است که نیمی یکنواخت و نیمی متمرکز حول  $x = 0$  انتخاب شدند (شکل ۴-۵)، زیرا جمله شامل  $\lambda_2$  (پخش) عملاً فقط در ناحیه جبهه اثر قابل توجه دارد و بدون نمونه‌برداری کافی از آن ناحیه، شناسایی



شکل ۴-۴: نقشه خطای مطلق بین جواب دقیق و پیش‌بینی شبکه در دامنه زمانی - مکانی؛ خطا عمدتاً در لایه باریک جبهه موج متمرکز است

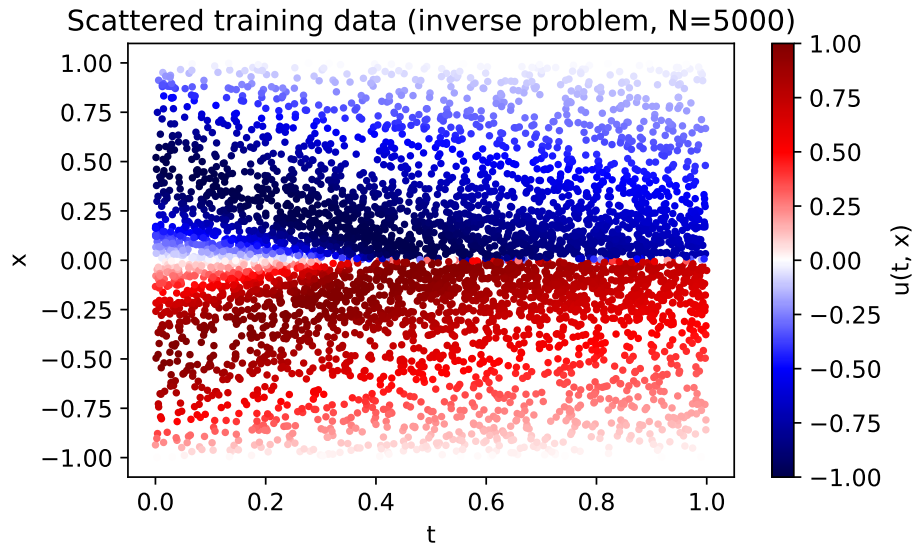
جدول ۲-۴: مقایسه نتیجه مسئله مستقیم با مقاله مرجع

| منبع           | معماری شبکه     | تعداد داده | نقاط هم‌مکانی | خطای نسبی $L_2$       |
|----------------|-----------------|------------|---------------|-----------------------|
| مقاله مرجع [۱] | ۹ لایه، ۲۰ نرون | ۱۰۰        | ۱۰۰۰۰         | $6.7 \times 10^{-4}$  |
| این پایان‌نامه | ۴ لایه، ۵۰ نرون | ۲۰۰        | ۱۰۰۰۰         | $2.15 \times 10^{-2}$ |

این ضریب ممکن نیست. شبکه مورد استفاده ۵ لایه پنهان با ۳۰ نرون در هر لایه دارد و پارامترهای  $\lambda_1$  و  $\lambda_2$  از مقدار اولیه صفر شروع به آموزش کردند. آموزش مانند مسئله مستقیم دومرحله‌ای (۶۰۰۰ گام آدام و سپس L-BFGS) انجام شد.

دو سناریو آزموده شد: داده تمیز (بدون نویز) و داده آلوده به نویز گاوسی یک درصد. نتایج در جدول ۳-۴ خلاصه شده است. در حالت بدون نویز، ضریب  $\lambda_1$  برابر ۰.۹۱۲ (با خطای ۸.۸٪) و ضریب  $\lambda_2$  برابر  $4.22 \times 10^{-3}$  (با خطای ۳۲.۶٪) شناسایی شد؛ مقادیر واقعی به ترتیب ۱ و  $0.00318 \approx 0.01/\pi$  هستند. در حالت نویزی نیز  $\lambda_1$  برابر ۰.۹۲۳ (خطای ۷.۷٪) و  $\lambda_2$  برابر  $3.97 \times 10^{-3}$  (خطای ۲۴.۹٪) به دست آمد.

همان‌طور که دیده می‌شود، ضریب انتقال ( $\lambda_1$ ) در هر دو سناریو با دقت قابل‌قبولی شناسایی شده، اما خطای ضریب پخش ( $\lambda_2$ ) به مراتب بیشتر است. این یافته با گزارش‌های مقاله مرجع نیز سازگار است، زیرا در آنجا نیز خطای  $\lambda_2$  همواره بزرگ‌تر از خطای  $\lambda_1$  گزارش شده است. دلیل این امر، کوچکی مقدار  $\lambda_2$  و موضعی بودن اثر آن (محدود به لایه باریک جبهه) است که شناسایی دقیق آن را ذاتاً به مسئله‌ای حساس تبدیل می‌کند؛ در بخش بعد بیشتر به این موضوع می‌پردازیم.



شکل ۴-۵: توزیع ۵۰۰۰ نقطه مشاهده پراکنده مورد استفاده در مسئله معکوس؛ رنگ نقاط نشان‌دهنده مقدار جواب است

جدول ۴-۳: نتایج شناسایی پارامترهای معادله برگرز در مسئله معکوس

| سناریو       | $\lambda_1$ شناسایی شده | خطای $\lambda_1$ | $\lambda_2$ شناسایی شده    | خطای $\lambda_2$ |
|--------------|-------------------------|------------------|----------------------------|------------------|
| بدون نویز    | 0.912                   | 8.8%             | $4.22 \times 10^{-3}$      | 32.6%            |
| نویز یک درصد | 0.923                   | 7.7%             | $3.97 \times 10^{-3}$      | 24.9%            |
| مقدار واقعی  | 1                       |                  | $0.01/\pi \approx 0.00318$ |                  |

## ۴-۵ بحث و تحلیل نتایج

**کارایی داده‌ای.** مهم‌ترین نقطه قوت مشاهده‌شده، کارایی داده‌ای روش است: در مسئله مستقیم، تنها با ۲۰۰ نقطه داده روی شرط اولیه و مرز (در برابر بیش از یکصد هزار نقطه شبکه مرجع)، جواب کل دامنه بازسازی شد. این ویژگی دقیقاً همان چیزی است که شبکه‌های آگاه از فیزیک را از مدل‌های صرفاً داده‌محور متمایز می‌کند و در کاربردهایی که داده تجربی کمیاب و گران است اهمیت زیادی دارد.

**چالش جبهه تند.** متمرکز شدن خطا در ناحیه جبهه موج نشان می‌دهد که گلوگاه اصلی دقت، تفکیک‌پذیری لایه‌های تند جواب است. تمرکز نیمی از نقاط هم‌مکانی در این ناحیه کمک قابل‌توجهی به بهبود نتیجه کرد، اما همچنان دقیق‌ترین بخش جواب همان ناحیه باقی ماند. این مشاهده با ادبیات موضوع سازگار است و راهکارهایی مانند وزن‌دهی تطبیقی جملات تابع هزینه یا نمونه‌برداری تطبیقی نقاط در پژوهش‌های بعدی پیشنهاد شده‌اند.

**حساسیت شناسایی ضریب پخش.** خطای بیشتر  $\lambda_2$  نسبت به  $\lambda_1$  ریشه فیزیکی دارد: جمله پخش  $\lambda_2 u_{xx}$  به سبب کوچکی  $\lambda_2$ ، تقریباً در همه دامنه (به جز لایه باریک

جبهه) در برابر جمله انتقال قابل چشم‌پوشی است؛ در نتیجه سیگنال آموزشی موجود برای تعیین دقیق آن ضعیف است. هر خطای کوچکی در بازسازی انحنای جواب در ناحیه جبهه نیز مستقیماً به خطای  $\lambda_2$  تبدیل می‌شود. با این حال، مرتبه بزرگی ضریب به‌درستی شناسایی شد و مقدار آن در هر دو سناریو در محدوده قابل‌قبولی از مقدار واقعی قرار گرفت.

**پایداری در برابر نویز.** مقایسه دو سناریوی تمیز و نویزی نشان می‌دهد که افزودن یک درصد نویز به داده‌ها، دقت شناسایی را به‌طور چشمگیری تخریب نکرده است. دلیل این پایداری آن است که مشتق‌های لازم برای تشکیل باقیمانده، به‌جای تفاضل محدود روی داده‌های نویزی، با مشتق‌گیری خودکار از روی شبکه‌ای هموار محاسبه می‌شوند و شبکه در عمل نقش یک فیلتر هموارساز را نیز ایفا می‌کند.

**مقایسه با روش‌های کلاسیک.** در مقایسه با حل‌گر مرجع طیفی که جوابی بسیار دقیق ولی فقط برای یک شرط اولیه و یک مجموعه پارامتر خاص تولید می‌کند، شبکه آموزش‌دیده تابعی پیوسته و مشتق‌پذیر از جواب در اختیار می‌گذارد که ارزیابی آن در هر نقطه دلخواه (حتی خارج از شبکه) ارزان است. از سوی دیگر، هزینه آموزش شبکه به‌مراتب بیشتر از یک اجرای حل‌گر کلاسیک است و دقت نهایی آن نیز کمتر است. بنابراین در شرایط فعلی، مزیت اصلی این چارچوب نه جایگزینی حل‌گرهای کلاسیک در مسائل مستقیم استاندارد، بلکه توانایی آن در مسائل معکوس، مسائل با داده ناقص و ترکیب داده و مدل فیزیکی است.

**محدودیت‌ها.** دامنه بخش عملی این پژوهش محدود به معادله برگرز یک‌بعدی و مدل زمان‌پیوسته بود و به‌سبب محدودیت توان محاسباتی، امکان جست‌وجوی گسترده ابرپارامترها (معماری شبکه، تعداد نقاط، راهبرد بهینه‌سازی) وجود نداشت. همچنین برآورد عدم قطعیت، که از مزیت‌های بالقوه روش‌های یادگیری ماشین است، در این پژوهش بررسی نشد.

## ۴-۶ جمع‌بندی فصل

در این فصل، پیاده‌سازی مدل زمان‌پیوسته شبکه آگاه از فیزیک برای معادله برگرز ارائه شد. یک حل‌گر مرجع طیفی - نمایی توسعه یافت و با دو آزمون مستقل راستی‌آزمایی گردید. در مسئله مستقیم، جواب کل دامنه تنها با ۲۰۰ نقطه داده اولیه و مرزی بازسازی شد و خطای نسبی  $2.15 \times 10^{-2}$  به دست آمد. در مسئله معکوس، ضرایب معادله از روی ۵۰۰۰ مشاهده پراکنده شناسایی شدند و اثر نویز بر دقت شناسایی بررسی گردید. تحلیل نتایج نشان داد که گلوگاه اصلی، تفکیک لایه تند جبهه موج و حساسیت ذاتی

شناسایی ضریب کوچک پخش است.

## فصل ۵

### نتیجه‌گیری و پیشنهادها

در این فصل، ابتدا خلاصه‌ای از آنچه در این پایان‌نامه انجام شد ارائه می‌شود، سپس یافته‌های اصلی و نتیجه‌گیری نهایی بیان می‌گردد و در پایان، پیشنهادهایی برای ادامه پژوهش در این زمینه مطرح می‌شود.

#### ۵-۱ خلاصه پژوهش

این پایان‌نامه به بررسی چارچوب شبکه‌های عصبی آگاه از فیزیک برای حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی اختصاص داشت. در فصل اول، موضوع، مسئله، اهداف و سؤال‌های پژوهش تعریف شد. در فصل دوم، مبانی نظری لازم شامل معادلات دیفرانسیل جزئی، روش‌های عددی کلاسیک، شبکه‌های عصبی و آموزش آن‌ها، مشتق‌گیری خودکار و پیشینه پژوهش در زمینه یادگیری ماشین آگاه از فیزیک مرور گردید. در فصل سوم، چارچوب اصلی پژوهش یعنی تعریف شبکه باقیمانده، طراحی تابع هزینه ترکیبی، مدل‌های زمان‌پیوسته و زمان‌گسسته و فرمول‌بندی مسئله معکوس تشریح شد. در فصل چهارم، مدل زمان‌پیوسته برای معادله برگرز یک‌بعدی با زبان پایتون و کتابخانه جکس پیاده‌سازی شد؛ یک حل‌گر مرجع مستقل بر پایه روش طیفی فوریه و گام‌زنی ETDRK4 توسعه یافت و با دو آزمون مستقل راستی‌آزمایی گردید؛ و نتایج حل مستقیم و کشف پارامترها گزارش و تحلیل شد.

#### ۵-۲ یافته‌ها و نتیجه‌گیری

یافته‌های اصلی این پژوهش را می‌توان به شرح زیر خلاصه کرد:

۱. مدل زمان‌پیوسته شبکه آگاه از فیزیک توانست جواب معادله برگرز را در کل دامنه زمانی - مکانی، تنها با ۲۰۰ نقطه داده اولیه و مرزی بازسازی کند و به خطای نسبی  $2.15 \times 10^{-2}$  در نرم  $L_2$  نسبت به حل مرجع دست یابد. این نتیجه، کارایی داده‌ای بالای این چارچوب را تأیید می‌کند.



۲. خطای جواب تقریباً به طور کامل در لایه باریک جبهه موج متمرکز بود و در بقیه دامنه ناچیز به دست آمد. تمرکز نیمی از نقاط هم‌مکانی در ناحیه جبهه، نقش مهمی در دستیابی به این نتیجه داشت و نشان داد که توزیع نقاط هم‌مکانی باید با فیزیک مسئله (محل گرادیان‌های تند) هماهنگ باشد.

۳. در مسئله معکوس، ضریب انتقال ( $\lambda_1$ ) با خطای 8.8% و ضریب پخش ( $\lambda_2$ ) با خطای 32.6% شناسایی شد. خطای بیشتر ضریب پخش، ریشه در کوچکی مقدار آن و موضعی بودن اثرش در لایه جبهه دارد و با یافته‌های مقاله مرجع سازگار است.

۴. افزودن یک درصد نویز به داده‌های مسئله معکوس، دقت شناسایی را به طور چشمگیری تخریب نکرد (خطای 7.7% برای  $\lambda_1$  و 24.9% برای  $\lambda_2$ )، که پایداری قابل قبول روش در برابر نویز را نشان می‌دهد.

۵. مقایسه با مقاله مرجع نشان داد که با امکانات محدود (پردازنده مرکزی و بودجه محاسباتی کمتر) می‌توان رفتار کیفی روش را به طور کامل بازتولید کرد، هرچند رسیدن به دقت عددی گزارش شده در مقاله نیازمند توان محاسباتی بیشتر و تنظیم دقیق‌تر ابرپارامترهاست.

در مجموع می‌توان نتیجه گرفت که شبکه‌های عصبی آگاه از فیزیک، چارچوبی ساده، کلی و کم‌داده برای ترکیب دانش فیزیکی با یادگیری ماشین ارائه می‌دهند و به‌ویژه در مسائلی که داده تجربی کمیاب است یا پارامترهای مدل مجهول‌اند، می‌توانند مکمل ارزشمندی برای روش‌های عددی کلاسیک باشند. در عین حال، این روش‌ها هنوز با چالش‌هایی مانند تفکیک لایه‌های تند جواب، هزینه بالای آموزش نسبت به یک اجرای حل‌گر کلاسیک و نبود تضمین‌های نظری همگرایی مواجه‌اند و نباید آن‌ها را جایگزینی بی‌قیدوشرط برای روش‌های کلاسیک دانست.

## ۳-۵ پیشنهادها برای پژوهش‌های آینده

در ادامه این پژوهش، موضوع‌های زیر پیشنهاد می‌شود:

۱. پیاده‌سازی مدل زمان‌گسسته با طرح‌های رانگ - کوتای ضمنی و مقایسه کارایی و دقت آن با مدل زمان‌پیوسته روی همان مسئله برگرز؛
۲. اعمال روش به معادلات دیگر مانند معادله گرما، معادله شرودینگر غیرخطی یا معادلات ناویر - استوکس در هندسه‌های ساده؛
۳. بررسی راهبردهای بهبود دقت در ناحیه جبهه، از جمله وزن‌دهی تطبیقی جملات

- تابع هزینه و نمونه برداری تطبیقی نقاط هم‌مکانی در حین آموزش؛
۴. مطالعه نظام‌مند اثر معماری شبکه (عمق و پهنا)، تعداد نقاط داده و هم‌مکانی و راهبرد بهینه‌سازی بر دقت نهایی؛
۵. ترکیب چارچوب آگاه از فیزیک با روش‌های برآورد عدم قطعیت، مانند شبکه‌های بیزی یا روش‌های مبتنی بر اجماع، برای کاربردهای حساس به اطمینان؛
۶. بررسی مسائل دوبرخی و ارزیابی مقیاس‌پذیری روش با افزایش ابعاد مسئله.

# Bibliography

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [2] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [3] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Machine learning of linear differential equations using Gaussian processes,” *Journal of Computational Physics*, vol. 348, pp. 683–693, 2017.
- [4] M. Raissi and G. E. Karniadakis, “Hidden physics models: Machine learning of nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 357, pp. 125–141, 2018.
- [5] A. Karpatne, G. Atluri, J. Faghmous, M. Steinbach, A. Banerjee, A. Ganguly, S. Shekhar, N. Samatova, and V. Kumar, “Theory-guided data science: A new paradigm for scientific discovery from data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 10, pp. 2318–2331, 2017.
- [6] M. Schmidt and H. Lipson, “Distilling free-form natural laws from experimental data,” *Science*, vol. 324, pp. 81–85, 2009.
- [7] S. L. Brunton, J. L. Proctor, and J. N. Kutz, “Discovering governing equations from data by sparse identification of nonlinear dynamical systems,” *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016.
- [8] S. Cuomo, V. S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, and F. Piccialli, “Scientific machine learning through physics-informed neural networks: Where we are and what’s next,” *Journal of Scientific Computing*, vol. 92, article 88, 2022.
- [9] G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989.
- [10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.

- [11] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016.
- [12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proceedings of the International Conference on Learning Representations (ICLR), 2015.
- [13] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” Mathematical Programming, vol. 45, pp. 503–528, 1989.
- [14] X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in Proceedings of AIS-TATS, pp. 249–256, 2010.
- [15] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic differentiation in machine learning: A survey,” Journal of Machine Learning Research, vol. 18, pp. 1–43, 2018.
- [16] R. J. LeVeque, Finite Difference Methods for Ordinary and Partial Differential Equations. SIAM, 2007.
- [17] L. C. Evans, Partial Differential Equations, 2nd ed. American Mathematical Society, 2010.
- [18] E. Hopf, “The partial differential equation  $u_t + uu_x = \mu u_{xx}$ ,” Communications on Pure and Applied Mathematics, vol. 3, pp. 201–230, 1950.
- [19] E. R. Benton and G. W. Platzman, “A table of solutions of the one-dimensional Burgers equation,” Quarterly of Applied Mathematics, vol. 30, pp. 195–212, 1972.
- [20] L. N. Trefethen, Spectral Methods in MATLAB. SIAM, 2000.
- [21] A.-K. Kassam and L. N. Trefethen, “Fourth-order time-stepping for stiff PDEs,” SIAM Journal on Scientific Computing, vol. 26, no. 4, pp. 1214–1233, 2005.
- [22] J. C. Butcher, Numerical Methods for Ordinary Differential Equations, 3rd ed. Wiley, 2016.
- [23] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” in Proceedings of OSDI, 2016.
- [24] J. Bradbury et al., “JAX: Composable transformations of Python+NumPy programs,” 2018. [Online]. Available: [github.com/google/jax](https://github.com/google/jax)

## واژه‌نامه

### واژه‌نامه فارسی به انگلیسی

| فارسی                   | انگلیسی                         |
|-------------------------|---------------------------------|
| شبکه عصبی آگاه از فیزیک | Physics-Informed Neural Network |
| معادله دیفرانسیل جزئی   | Partial Differential Equation   |
| معادله دیفرانسیل معمولی | Ordinary Differential Equation  |
| مسئله مستقیم            | Forward Problem                 |
| مسئله معکوس             | Inverse Problem                 |
| مشتق‌گیری خودکار        | Automatic Differentiation       |
| پس‌انتشار               | Backpropagation                 |
| تابع هزینه              | Loss Function                   |
| میانگین مربعات خطا      | Mean Squared Error              |
| نقاط هم‌مکانی           | Collocation Points              |
| شرط اولیه               | Initial Condition               |
| شرط مرزی                | Boundary Condition              |
| تابع فعال‌ساز           | Activation Function             |
| نرخ یادگیری             | Learning Rate                   |
| بیش‌برازش               | Overfitting                     |
| نفرین ابعاد             | Curse of Dimensionality         |

## واژه‌نامه انگلیسی به فارسی

| فارسی                   | انگلیسی                         |
|-------------------------|---------------------------------|
| تابع فعال‌ساز           | Activation Function             |
| مشتق‌گیری خودکار        | Automatic Differentiation       |
| پس‌انتشار               | Backpropagation                 |
| شرط مرزی                | Boundary Condition              |
| نقاط هم‌مکانی           | Collocation Points              |
| نفرین ابعاد             | Curse of Dimensionality         |
| مسئله مستقیم            | Forward Problem                 |
| شرط اولیه               | Initial Condition               |
| مسئله معکوس             | Inverse Problem                 |
| نرخ یادگیری             | Learning Rate                   |
| تابع هزینه              | Loss Function                   |
| میانگین مربعات خطا      | Mean Squared Error              |
| معادله دیفرانسیل معمولی | Ordinary Differential Equation  |
| بیش‌برازش               | Overfitting                     |
| معادله دیفرانسیل جزئی   | Partial Differential Equation   |
| شبکه عصبی آگاه از فیزیک | Physics-Informed Neural Network |

## پیوست الف: کد پیاده‌سازی

در این پیوست، کد کامل پیاده‌سازی بخش عملی پایان‌نامه آورده شده است. این کد با زبان پایتون و کتابخانه جکس نوشته شده و شامل سه بخش اصلی است: حل‌گر مرجع طیفی برای تولید جواب دقیق معادله برگرز، پیاده‌سازی مدل زمان‌پیوسته برای مسئله مستقیم همراه با آموزش دومرحله‌ای آدام و L-BFGS، و پیاده‌سازی مسئله معکوس برای شناسایی پارامترهای معادله. اجرای این کد علاوه بر آموزش شبکه‌ها، همه شکل‌های فصل چهارم را نیز تولید می‌کند. نسخه الکترونیکی این فایل همراه با راهنمای اجرا در پوشه code موجود است.

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 Physics-Informed Neural Network (PINN) for the 1D Burgers equation.
5
6 Reproduction (for B.Sc. thesis) of the continuous-time results of:
7 Raissi, Perdikaris, Karniadakis, "Physics-informed neural networks
8   ...",
9   J. Comput. Phys. 378 (2019) 686-707. (Appendix A.1 and B.1)
10
11 Two modes:
12   forward : data-driven solution of Burgers (learn u(t,x) from IC/BC
13             + PDE residual)
14   inverse  : data-driven discovery of Burgers (learn lambda1, lambda2
15             from scattered data)
16
17 Burgers equation:  $u_t + \lambda_1 * u * u_x - \lambda_2 * u_{xx} = 0$ 
18   x in [-1, 1], t in [0, 1], lambda1 = 1.0, lambda2 = 0.01/pi (
19             forward)
20   u(0, x) = -sin(pi*x), u(t, -1) = u(t, 1) = 0
21
22 Reference solution: Fourier spectral in space (N=512, 2/3 de-aliasing
23                     ) + RK4 in time.
24
25 Requirements: jax[cpu], numpy, scipy, matplotlib
```

```

21 Output: metrics JSON + figures written to ../figs/
22 """
23 import argparse
24 import json
25 import os
26 import time
27
28 import numpy as np
29
30 import jax
31 from jax import grad, jit, vmap
32 from jax.tree_util import tree_map
33 from jax.flatten_util import ravel_pytree
34 import jax.numpy as jnp
35 from jax import random
36
37 try:
38     jax.config.update("jax_enable_x64", True)
39 except Exception: # very old jax
40     from jax import config as _cfg
41     _cfg.update("jax_enable_x64", True)
42
43 import matplotlib
44 matplotlib.use("Agg")
45 import matplotlib.pyplot as plt
46
47 HERE = os.path.dirname(os.path.abspath(__file__))
48 FIGDIR = os.path.join(os.path.dirname(HERE), "figs")
49 os.makedirs(FIGDIR, exist_ok=True)
50
51 PI = float(np.pi)
52 NU = 0.01 / PI # viscosity used in the paper
53
54
55 #
56 # -----
57
58 # Reference spectral solver (independent of the neural network)
59 #

```



---

```

58 def burgers_reference(nx=1024, dt=0.001, t_out=None, nu=NU):
59     """Solve Burgers with Fourier spectral + ETD RK4 (Kassam &
60         Trefethen 2005).
61
62     The diffusion term is treated exactly in Fourier space, so the
63     scheme
64     stays stable for the stiff thin shock layer. Returns t, x, U[nt,
65     nx].
66     """
67     L = 2.0
68     x = np.linspace(-1.0, 1.0, nx, endpoint=False)
69     k = 2.0 * PI * np.fft.fftfreq(nx, d=L / nx)
70     dealias = (np.abs(k) < (nx // 3) * (2.0 * PI / L)).astype(float)
71
72     # ETD RK4 coefficients via contour integral (M points on unit
73     circle)
74     M = 32
75     r = np.exp(1j * PI * (np.arange(1, M + 1) - 0.5) / M)
76     Lv = -nu * k ** 2
77     LR = dt * Lv[:, None] + r[None, :]
78     E = np.exp(dt * Lv)
79     E2 = np.exp(dt * Lv / 2.0)
80     Q = dt * np.real(np.mean((np.exp(LR / 2.0) - 1.0) / LR, axis=1))
81     f1 = dt * np.real(np.mean(
82         (-4.0 - LR + np.exp(LR) * (4.0 - 3.0 * LR + LR ** 2)) / LR **
83         3, axis=1))
84     f2 = dt * np.real(np.mean(
85         2.0 * (2.0 + LR + np.exp(LR) * (-2.0 + LR)) / LR ** 3, axis
86         =1))
87     f3 = dt * np.real(np.mean(
88         (-4.0 - 3.0 * LR - LR ** 2 + np.exp(LR) * (4.0 - LR)) / LR **
89         3, axis=1))
90
91     def nlin(uu):
92         uh = np.fft.fft(uu)
93         ux = np.real(np.fft.ifft(1j * k * uh))
94         return np.fft.fft(-uu * ux) * dealias

```

```

88
89     u = -np.sin(PI * x)
90     v = np.fft.fft(u)
91     if t_out is None:
92         t_out = np.linspace(0, 1.0, 101)
93     U = [u.copy()]
94     t = 0.0
95     for target in t_out[1:]:
96         nsteps = int(round((target - t) / dt))
97         for _ in range(nsteps):
98             Nv = nlin(np.real(np.fft.ifft(v)))
99             a = E2 * v + Q * Nv
100             Na = nlin(np.real(np.fft.ifft(a)))
101             b = E2 * v + Q * Na
102             Nb = nlin(np.real(np.fft.ifft(b)))
103             c = E2 * a + Q * (2.0 * Nb - Nv)
104             Nc = nlin(np.real(np.fft.ifft(c)))
105             v = E * v + f1 * Nv + f2 * (Na + Nb) + f3 * Nc
106             t += dt
107             U.append(np.real(np.fft.ifft(v)))
108     U = np.array(U)
109     assert np.all(np.isfinite(U)), "reference solver diverged!"
110     return t_out, x, U
111
112
113 #
114 -----
115
116 # MLP + derivatives (automatic differentiation)
117 #
118 -----
119
120 def init_mlp(key, layers):
121     keys = random.split(key, len(layers) - 1)
122     params = []
123     for kk, (din, dout) in zip(keys, zip(layers[:-1], layers[1:])):
124         W = random.normal(kk, (din, dout)) * np.sqrt(2.0 / (din +
125             dout))
126         b = jnp.zeros((dout,))

```

```

122         params.append((W, b))
123     return params
124
125
126 def u_point(params, t, x):
127     h = jnp.stack([t, x])
128     for W, b in params[:-1]:
129         h = jnp.tanh(jnp.dot(h, W) + b)
130     W, b = params[-1]
131     return jnp.dot(h, W)[0] + b[0]
132
133
134 _u_t = grad(u_point, argnums=1)
135 _u_x = grad(u_point, argnums=2)
136 _u_xx = grad(grad(u_point, argnums=2), argnums=2)
137
138 u_pred = jit(vmap(u_point, in_axes=(None, 0, 0)))
139 u_t_pred = jit(vmap(_u_t, in_axes=(None, 0, 0)))
140 u_x_pred = jit(vmap(_u_x, in_axes=(None, 0, 0)))
141 u_xx_pred = jit(vmap(_u_xx, in_axes=(None, 0, 0)))
142
143
144 #
145 -----
146
147 # Manual Adam (no extra dependencies, works with any jax version)
148 #
149 -----
150
151 def adam(loss_grad_fn, params, iters, lr=1e-3, log_every=500, tag="
adam"):
152     m = tree_map(jnp.zeros_like, params)
153     v = tree_map(jnp.zeros_like, params)
154     b1, b2, eps = 0.9, 0.999, 1e-8
155     hist = []
156     for it in range(1, iters + 1):
157         loss, g = loss_grad_fn(params)
158         m = tree_map(lambda mm, gg: b1 * mm + (1 - b1) * gg, m, g)
159         v = tree_map(lambda vv, gg: b2 * vv + (1 - b2) * (gg ** 2), v

```

```

    , g)
156     mh = tree_map(lambda mm: mm / (1 - b1 ** it), m)
157     vh = tree_map(lambda vv: vv / (1 - b2 ** it), v)
158     params = tree_map(lambda p, mm, vv: p - lr * mm / (jnp.sqrt(
        vv) + eps),
159                        params, mh, vh)
160     hist.append(float(loss))
161     if it % log_every == 0 or it == 1:
162         print(f"[{tag}] iter {it:5d}/{iters} loss = {float(loss)
            :.6e}", flush=True)
163     return params, hist
164
165
166 def lbfgs(loss_fn, grad_fn, params, maxfun=2000):
167     from scipy.optimize import minimize
168     x0, unravel = ravel_pytree(params)
169     x0 = np.asarray(x0, dtype=np.float64)
170
171     def fun(x):
172         return float(loss_fn(unravel(jnp.asarray(x))))
173
174     def jac(x):
175         g = grad_fn(unravel(jnp.asarray(x)))
176         flat, _ = ravel_pytree(g)
177         return np.asarray(flat, dtype=np.float64)
178
179     state = {"n": 0, "hist": []}
180
181     def cb(x):
182         state["n"] += 1
183         if state["n"] % 200 == 0:
184             print(f"[lbfgs] eval {state['n']} loss = {fun(x):.6e}",
                flush=True)
185
186     t0 = time.time()
187     res = minimize(fun, x0, jac=jac, method="L-BFGS-B",
188                  callback=cb,
189                  options={"maxfun": maxfun, "maxiter": maxfun,
190                          "ftol": 1e-12, "gtol": 1e-10})

```

```

191     print(f"[lbfgs] done: {res.message}  fun={res.fun:.6e}  "
192           f"nit={res.nit}  nfev={res.nfev}  time={time.time()-t0:.1f}s
193           ", flush=True)
194
195     return unravel(jnp.asarray(res.x, dtype=jnp.float64)), float(res.
196                    fun)
197
198 #
199 -----
200
201 # Forward problem
202 #
203 -----
204
205 def run_forward(adam_iters=6000, lbfgs_maxfun=6000, seed=0):
206     rng = np.random.RandomState(seed)
207     print("== forward: solving reference ==", flush=True)
208     t_g, x_g, Uref = burgers_reference()
209     print(f"    reference grid: t={t_g.shape}, x={x_g.shape}, "
210           f"|u|max at t=1: {np.abs(Uref[-1]).max():.4f}", flush=True)
211
212     # Training data: Nu points on initial + boundary
213     Nu0, Nub = 100, 100
214     x0 = rng.uniform(-1, 1, Nu0)
215     t0 = np.zeros_like(x0)
216     u0 = -np.sin(PI * x0)
217     tb = rng.uniform(0, 1, Nub)
218     xb = rng.choice([-1.0, 1.0], size=Nub)
219     ub = np.zeros_like(tb)
220     t_u = np.concatenate([t0, tb])
221     x_u = np.concatenate([x0, xb])
222     u_u = np.concatenate([u0, ub])
223
224     # Collocation points: half uniform + half clustered around the
225         shock (x=0),
226
227     # where the solution develops a steep front for t > ~0.3.
228     Nf1, Nf2 = 5000, 5000
229     t_f1 = rng.uniform(0, 1, Nf1)
230     x_f1 = rng.uniform(-1, 1, Nf1)

```

```

223 t_f2 = rng.uniform(0, 1, Nf2)
224 x_f2 = np.clip(rng.normal(0.0, 0.2, Nf2), -1.0, 1.0)
225 t_f = np.concatenate([t_f1, t_f2])
226 x_f = np.concatenate([x_f1, x_f2])
227 Nf = Nf1 + Nf2
228
229 t_u_ = jnp.asarray(t_u); x_u_ = jnp.asarray(x_u); u_u_ = jnp.
    asarray(u_u)
230 t_f_ = jnp.asarray(t_f); x_f_ = jnp.asarray(x_f)
231
232 layers = [2, 50, 50, 50, 50, 1]
233 params = init_mlp(random.PRNGKey(seed), layers)
234 npar = sum(w.size + b.size for w, b in params)
235 print(f"== forward: network {layers} params={npar} "
236       f"Nu={len(t_u)} Nf={Nf} ==", flush=True)
237
238 @jit
239 def loss_fn(p):
240     mse_u = jnp.mean((u_pred(p, t_u_, x_u_) - u_u_) ** 2)
241     uu = u_pred(p, t_f_, x_f_)
242     f = (u_t_pred(p, t_f_, x_f_) + uu * u_x_pred(p, t_f_, x_f_)
243         - NU * u_xx_pred(p, t_f_, x_f_))
244     mse_f = jnp.mean(f ** 2)
245     return mse_u + mse_f
246
247 @jit
248 def grad_fn(p):
249     return grad(loss_fn)(p)
250
251 def loss_grad(p):
252     return loss_fn(p), grad_fn(p)
253
254 print(f"[forward] initial loss = {float(loss_fn(params)):.6e}",
255       flush=True)
256 t_start = time.time()
257 params, hist_adam = adam(loss_grad, params, adam_iters, lr=1e-3,
258                          log_every=500, tag="forward/adam")
259 params, loss_lbfgs = lbfgs(loss_fn, grad_fn, params, maxfun=
260                          lbfgs_maxfun)

```

```

259     print(f"[forward] total time = {time.time()-t_start:.1f}s", flush
          =True)
260     mse_u_fin = float(jnp.mean((u_pred(params, t_u_, x_u_) - u_u_) **
          2))
261     _uu = u_pred(params, t_f_, x_f_)
262     _ff = (u_t_pred(params, t_f_, x_f_) + _uu * u_x_pred(params, t_f_
          , x_f_)
          - NU * u_xx_pred(params, t_f_, x_f_))
263     mse_f_fin = float(jnp.mean(_ff ** 2))
264     print(f"[forward] final MSE_u = {mse_u_fin:.6e}, MSE_f = {
          mse_f_fin:.6e}",
          flush=True)
265
266
267
268     # Evaluate on the full reference grid
269     TT, XX = np.meshgrid(t_g, x_g, indexing="ij")
270     Up = np.asarray(u_pred(params, jnp.asarray(TT.ravel()),
          jnp.asarray(XX.ravel()))).reshape(TT.shape
          )
271
272     err = np.linalg.norm(Up - Uref) / np.linalg.norm(Uref)
273     print(f"[forward] relative L2 error = {err:.6e}", flush=True)
274
275     metrics = {
276         "mode": "forward",
277         "layers": layers, "n_params": int(npar),
278         "Nu": int(len(t_u)), "Nf": int(Nf),
279         "adam_iters": adam_iters, "lbfgs_maxfun": lbfgs_maxfun,
280         "loss_adam_final": hist_adam[-1], "loss_lbfgs_final":
          loss_lbfgs,
281         "mse_u_final": mse_u_fin, "mse_f_final": mse_f_fin,
282         "rel_l2": float(err), "seed": seed,
283         "nu": NU,
284     }
285     with open(os.path.join(FIGDIR, "forward_metrics.json"), "w") as f
          :
286         json.dump(metrics, f, indent=2)
287     np.savez(os.path.join(FIGDIR, "forward_data.npz"),
288             t=t_g, x=x_g, Uref=Uref, Upred=Up,
289             hist_adam=np.array(hist_adam),
290             t_u=t_u, x_u=x_u)

```

```

291
292 # --- figures ---
293 plt.rcParams.update({"font.size": 11})
294 # 1) snapshots
295 fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.2), sharey=True)
296 for ax, tc in zip(axes, [0.25, 0.5, 0.75]):
297     i = int(np.argmin(np.abs(t_g - tc)))
298     ax.plot(x_g, Uref[i], "k-", lw=1.6, label="Exact")
299     ax.plot(x_g, Up[i], "r--", lw=1.3, label="PINN")
300     ax.set_title(f"t = {t_g[i]:.2f}")
301     ax.set_xlabel("x")
302     ax.grid(alpha=0.3)
303 axes[0].set_ylabel("u(t, x)")
304 axes[0].legend(frameon=True)
305 fig.suptitle("Burgers equation: exact vs PINN (forward problem)")
306 fig.tight_layout()
307 fig.savefig(os.path.join(FIGDIR, "burgers_snapshots.pdf"))
308 plt.close(fig)
309
310 # 2) loss history (raw curve + running minimum, as Adam is spiky)
311 fig, ax = plt.subplots(figsize=(6.5, 3.8))
312 hist_adam = np.asarray(hist_adam)
313 runmin = np.minimum.accumulate(hist_adam)
314 ax.semilogy(hist_adam, color="lightsteelblue", lw=0.8, label="
    Adam (raw)")
315 ax.semilogy(runmin, "b-", lw=1.4, label="Adam (running min)")
316 ax.axhline(loss_lbfgs, color="r", ls="--", lw=1.2, label="L-BFGS
    (final)")
317 ax.set_xlabel("Adam iteration")
318 ax.set_ylabel("Loss (MSE_u + MSE_f)")
319 ax.set_title("Training loss history (forward problem)")
320 ax.grid(alpha=0.3, which="both")
321 ax.legend()
322 fig.tight_layout()
323 fig.savefig(os.path.join(FIGDIR, "loss_history.pdf"))
324 plt.close(fig)
325
326 # 3) error heatmap
327 fig, ax = plt.subplots(figsize=(6.5, 3.8))

```



```

328     im = ax.imshow(np.abs(Up - Uref), origin="lower", aspect="auto",
329                     extent=[x_g[0], x_g[-1], t_g[0], t_g[-1]], cmap="
                        hot")
330     ax.set_xlabel("x")
331     ax.set_ylabel("t")
332     ax.set_title("Absolute error |u_exact - u_pinn| (forward problem)
                    ")
333     fig.colorbar(im, ax=ax, label="|error|")
334     fig.tight_layout()
335     fig.savefig(os.path.join(FIGDIR, "error_heatmap.pdf"))
336     plt.close(fig)
337
338     # 4) training points
339     fig, ax = plt.subplots(figsize=(6.2, 3.6))
340     ax.scatter(t_f[:,5], x_f[:,5], s=3, c="lightsteelblue", label="
        Collocation (subset)")
341     ax.scatter(t_u, x_u, s=14, c="red", marker="x", label="IC/BC data
        ")
342     ax.set_xlabel("t")
343     ax.set_ylabel("x")
344     ax.set_title("Training points (forward problem)")
345     ax.legend()
346     fig.tight_layout()
347     fig.savefig(os.path.join(FIGDIR, "forward_points.pdf"))
348     plt.close(fig)
349     print("[forward] figures saved.", flush=True)
350     return metrics
351
352
353     #
    -----
354 # Inverse problem (discovery of lambda1, lambda2)
355 #
    -----
356 def run_inverse(n_data=5000, noise=0.0, adam_iters=6000, lbfgs_maxfun
    =6000,
357                 seed=1):

```

```

358 rng = np.random.RandomState(seed)
359 print(f"== inverse: n={n_data}, noise={noise} ==", flush=True)
360 t_g, x_g, Uref = burgers_reference()
361 # scattered observations: half uniform + half clustered around
    the shock,
362 # where the u_xx term (hence lambda2) actually matters.
363 n1, n2 = n_data // 2, n_data - n_data // 2
364 ti = np.concatenate([rng.uniform(0, 1, n1), rng.uniform(0, 1, n2)
    ])
365 xi = np.concatenate([rng.uniform(-1, 1, n1),
366                     np.clip(rng.normal(0.0, 0.2, n2), -1.0, 1.0)
    ])
367 ii = np.clip((ti * (len(t_g) - 1)).astype(int), 0, len(t_g) - 1)
368 jj = np.clip(((xi + 1) / 2 * (len(x_g))).astype(int), 0, len(x_g)
    - 1)
369 ui = Uref[ii, jj].copy()
370 if noise > 0:
371     ui = ui + noise * np.std(ui) * rng.randn(n_data)
372
373 t_d = jnp.asarray(ti); x_d = jnp.asarray(xi); u_d = jnp.asarray(
    ui)
374
375 layers = [2, 30, 30, 30, 30, 30, 1]
376 params = init_mlp(random.PRNGKey(seed), layers)
377 lam = jnp.array([0.0, 0.0]) # lambda1, lambda2 to be identified
378 npar = sum(w.size + b.size for w, b in params) + 2
379 print(f"== inverse: network {layers} params={npar} ==", flush=
    True)
380
381 @jit
382 def loss_fn(pl):
383     p, l = pl
384     mse_u = jnp.mean((u_pred(p, t_d, x_d) - u_d) ** 2)
385     uu = u_pred(p, t_d, x_d)
386     f = (u_t_pred(p, t_d, x_d) + l[0] * uu * u_x_pred(p, t_d, x_d
    )
387         - l[1] * u_xx_pred(p, t_d, x_d))
388     return mse_u + jnp.mean(f ** 2)
389

```

```

390 @jit
391 def grad_fn(pl):
392     return grad(loss_fn)(pl)
393
394 def loss_grad(pl):
395     return loss_fn(pl), grad_fn(pl)
396
397 print(f"[inverse] initial loss = {float(loss_fn((params, lam)))
398       :.6e}", flush=True)
399 t_start = time.time()
400 (params, lam), hist_adam = adam(loss_grad, (params, lam),
401                                adam_iters,
402                                lr=1e-3, log_every=500, tag="
403                                inverse/adam")
404 (params, lam), loss_lbfgs = lbfgs(loss_fn, grad_fn, (params, lam)
405                                   ,
406                                   maxfun=lbfgs_maxfun)
407 print(f"[inverse] total time = {time.time()-t_start:.1f}s", flush
408       =True)
409
410 lam = np.asarray(lam, dtype=float)
411 true = np.array([1.0, NU])
412 pct = 100.0 * np.abs(lam - true) / np.abs(true)
413 print(f"[inverse] identified lambda = {lam}, true = {true}",
414       flush=True)
415 print(f"[inverse] percentage errors = {pct}%", flush=True)
416
417 metrics = {
418     "mode": "inverse", "layers": layers, "n_params": int(npar),
419     "n_data": n_data, "noise": noise,
420     "adam_iters": adam_iters, "lbfgs_maxfun": lbfgs_maxfun,
421     "loss_adam_final": float(hist_adam[-1]),
422     "loss_lbfgs_final": float(loss_lbfgs),
423     "lambda_identified": [float(lam[0]), float(lam[1])],
424     "lambda_true": [1.0, NU],
425     "pct_error": [float(pct[0]), float(pct[1])],
426     "seed": seed,
427 }
428 tag = f"inverse_metrics_noise{int(100*noise)}.json"

```

```

423     with open(os.path.join(FIGDIR, tag), "w") as f:
424         json.dump(metrics, f, indent=2)
425
426     # scatter figure (only for the clean case to keep the thesis
427         light)
428
429     if noise == 0.0:
430         fig, ax = plt.subplots(figsize=(6.2, 3.8))
431         sc = ax.scatter(ti, xi, s=6, c=ui, cmap="seismic", vmin=-1,
432             vmax=1)
433         ax.set_xlabel("t")
434         ax.set_ylabel("x")
435         ax.set_title(f"Scattered training data (inverse problem, N={
436             n_data})")
437         fig.colorbar(sc, ax=ax, label="u(t, x)")
438         fig.tight_layout()
439         fig.savefig(os.path.join(FIGDIR, "inverse_points.pdf"))
440         plt.close(fig)
441
442     return metrics
443
444 def main():
445     ap = argparse.ArgumentParser()
446     ap.add_argument("--mode", choices=["forward", "inverse", "all"],
447         default="all")
448     ap.add_argument("--adam-iters", type=int, default=6000)
449     ap.add_argument("--lbfgs-maxfun", type=int, default=6000)
450     args = ap.parse_args()
451
452     all_metrics = {}
453     if args.mode in ("forward", "all"):
454         all_metrics["forward"] = run_forward(
455             adam_iters=args.adam_iters, lbfgs_maxfun=args.
456                 lbfgs_maxfun)
457     if args.mode in ("inverse", "all"):
458         all_metrics["inverse_clean"] = run_inverse(
459             noise=0.0, adam_iters=args.adam_iters,
460             lbfgs_maxfun=args.lbfgs_maxfun)
461         all_metrics["inverse_noisy"] = run_inverse(
462             noise=0.01, adam_iters=args.adam_iters,

```

```
458         lbfgs_maxfun=args.lbfgs_maxfun, seed=2)
459     with open(os.path.join(FIGDIR, "metrics_all.json"), "w") as f:
460         json.dump(all_metrics, f, indent=2, default=str)
461     print("ALL DONE. metrics:", json.dumps(all_metrics, indent=2,
462         default=str),
463         flush=True)
464
465 if __name__ == "__main__":
466     main()
```

## فرم ارزیابی پایان نامه کارشناسی

|                          |                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------|
| نام و نام خانوادگی       | محمد امیر بابازاد                                                                      |
| شماره دانشجویی           | ۱۴۰۲۱۳۵۱۰۴                                                                             |
| رشته تحصیلی              | علوم کامپیوتر                                                                          |
| عنوان پایان نامه         | شبکه های عصبی آگاه از فیزیک برای حل مسائل مستقیم و معکوس معادلات دیفرانسیل جزئی غیرخطی |
| استاد راهنما             | دکتر بابک آذرنوید                                                                      |
| تاریخ دفاع               | .....                                                                                  |
| نمره به عدد              | .....                                                                                  |
| نمره به حروف             | .....                                                                                  |
| نام و امضای استاد راهنما | .....                                                                                  |
| نام و امضای داور         | .....                                                                                  |
| نام و امضای مدیر گروه    | .....                                                                                  |

# Abstract

Numerical solution of nonlinear partial differential equations is a fundamental problem in scientific computing. Classical numerical methods, while accurate and widely used, depend on computational meshes and struggle in high dimensions; on the other hand, purely data-driven deep learning models require large amounts of training data and offer no guarantee that their outputs respect the laws of physics. In recent years, the framework of physics-informed neural networks has been introduced as a way to combine physical knowledge (the governing equation of the system) with machine learning. The aim of this thesis is to study, implement, and evaluate this framework for solving forward and inverse problems involving nonlinear partial differential equations.

In this work, we first review the required theoretical background, including partial differential equations, classical numerical methods, neural networks, automatic differentiation, and the background of physics-aware machine learning. We then describe the physics-informed framework, including the definition of the residual network, the combined data-physics loss function, and the continuous- and discrete-time models. In the practical part, the continuous-time model is implemented for the one-dimensional Burgers equation in Python using the JAX library, and an independent reference solver based on the Fourier spectral method and fourth-order time stepping is developed to generate the exact solution and verified against the Cole-Hopf semi-analytical solution.

In the forward problem, the solution over the whole spatio-temporal domain is reconstructed using only 200 initial and boundary data points, achieving a relative error of  $2.15 \times 10^{-2}$  in the  $L_2$  norm; the error is mostly

concentrated in the thin shock layer. In the inverse problem, the equation coefficients are identified from 5000 scattered observations: the advection coefficient with an error of 8.8% and the diffusion coefficient with an error of 32.6% in the noise-free case, and 7.7% and 24.9% respectively under one percent noise. The results show that this framework achieves an acceptable solution with little data, is reasonably robust to noise, and that its main bottleneck is resolving the steep solution layers and the inherent sensitivity of identifying small coefficients.

**Keywords:** Physics-informed neural network, Nonlinear partial differential equation, Forward problem, Inverse problem, Burgers equation, Automatic differentiation



**University of Bonab**  
**Department of Computer Science**

**B.Sc. Thesis in Computer Science**

**Physics-Informed Neural Networks  
for Solving Forward and Inverse  
Problems  
Involving Nonlinear Partial  
Differential Equations**

**Supervisor: Dr. Babak Azarnavid**

**By: Mohammad Amir Babazadeh**

Student ID: 1402135104

**September 2026**